



Cloud Computing Lab 12

**Submitted To: Sir Waqas Saleem
Submitted By: Hamail Fatima
Roll No: 2023-BSE-023
Section: 5(A)**

Course Name: Cloud Computing

Lab 12 repo link: https://github.com/Hamail-maker/CC_Hamail-Fatima_2023-BSE-023_Lab12.git

Lab # 12

Task 0 Lab Setup (Codespace & GH CLI)

All actions below should be executed inside the Codespace shell.

Create Codespace & connect:

create or open codespace via GH CLI (example)

gh repo create CC_<YourName>_<YourRollNumber>/Lab12 --public

gh codespace create --repo <user_name>/Lab12

gh codespace list

gh codespace ssh -c <your_codespace_name>

- **Save screenshot as:** task0_codespace_create_and_list.png – output showing repo creation/codespace list.

```
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab12> gh codespace create --repo Hamail-maker/CC_Hamail-Fatima_2023-BSE-023_Lab12
✓ Codespaces usage for this repository is paid for by Hamail-maker
? Choose Machine Type: 2 cores, 8 GB RAM, 32 GB storage
humble-space-giggle-jjgx7p67xq7j2qx74
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab12> gh codespace list
```

NAME	DISPLAY NAME	REPOSITORY	BRANCH	STATE	CREATED AT
potential-eureka-q7xw45r4wg5vhpqxq	potential eureka	Hamail-maker/my-lab-repo	main*	Shutdown	about 17 days ago
reimagined-orbit-x54jgg6pjgg5hv9p	reimagined orbit	Hamail-maker/CC_hamail-maker_023_Lab11	main*	Shutdown	about 13 days ago
redesigned-space-telegram-v65xrj4rx7r5hv56	redesigned space telegram	Hamail-maker/cc-hamail-fatima-023	main	Shutdown	about 8 hours ago
ideal-waddle-jjgx7p67xg76fj659	ideal waddle	Hamail-maker/CC_Hamail-Fatima_2023-BSE-023	main	Shutdown	about 7 hours ago
humble-space-giggle-jjgx7p67xq7j2qx74	humble space giggle	Hamail-maker/CC_Hamail-Fatima_2023-BSE-023...	main	Available	less than a minute ago

```
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab12>
```

- **Save screenshot as:** task0_codespace_ssh_connected.png – terminal inside the Codespace shell after ssh.

```
PS C:\Users\admin\CC_Hamail-Fatima_2023-BSE-023_Lab12> gh codespace ssh -c humble-space-giggle-jjgx7p67xq7j2qx74
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

Task 1 – Organize Terraform code into separate files

In this task, you will split a monolithic Terraform configuration into separate, well-organized files following best practices.

1. Create the initial project structure:

```
mkdir -p ~/Lab12
```

```
cd ~/Lab12
```

- **Save screenshot as:** task1_project_directory.png – terminal showing directory creation.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ mkdir -p ~/Lab12
cd ~/Lab12
@Hamail-maker →~/Lab12 $
```

2. Create all required files:

```
touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars entry-script.sh
```

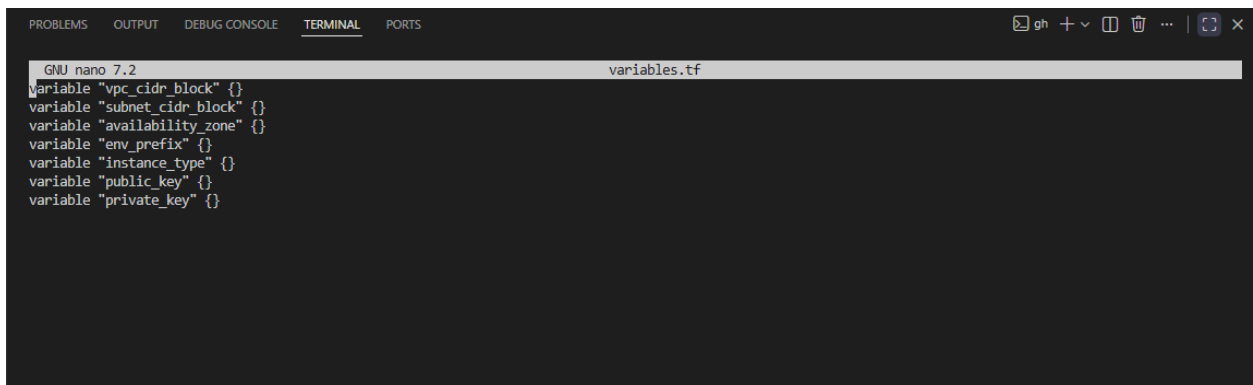
- **Save screenshot as:** task1_files_created.png – terminal showing all files created (use ls -la).

```
@Hamail-maker → ~/Lab12 $ touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars entry-script.sh
@Hamail-maker → ~/Lab12 $ ls -la
total 12
drwxrwxr-x 2 codespace codespace 4096 Jan  2 20:23 .
drwxr-x--- 1 codespace codespace 4096 Jan  2 20:21 ..
-rw-rw-r-- 1 codespace codespace   0 Jan  2 20:23 entry-script.sh
-rw-rw-r-- 1 codespace codespace   0 Jan  2 20:23 locals.tf
-rw-rw-r-- 1 codespace codespace   0 Jan  2 20:23 main.tf
-rw-rw-r-- 1 codespace codespace   0 Jan  2 20:23 outputs.tf
-rw-rw-r-- 1 codespace codespace   0 Jan  2 20:23 terraform.tfvars
-rw-rw-r-- 1 codespace codespace   0 Jan  2 20:23 variables.tf
@Hamail-maker → ~/Lab12 $
```

3. Create variables.tf with the following content:

```
variable "vpc_cidr_block" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}
variable "instance_type" {}
variable "public_key" {}
variable "private_key" {}
```

- **Save screenshot as:** task1_variables_tf.png – content of variables.tf file.

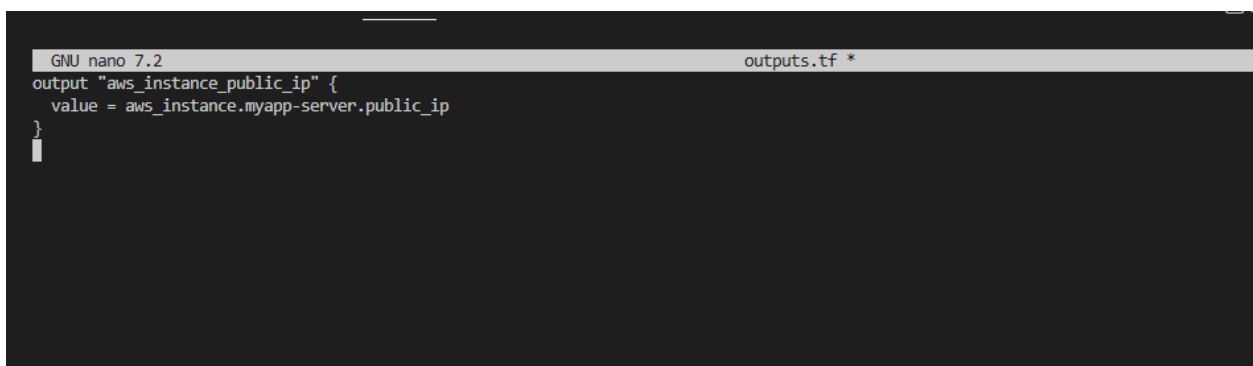
A screenshot of a terminal window with the title bar "PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS". The terminal shows the content of a file named "variables.tf" using the GNU nano 7.2 editor. The content of the file is a list of Terraform variables, each defined as a variable with an empty object as a default value.

```
GNU nano 7.2 variables.tf
variable "vpc_cidr_block" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}
variable "instance_type" {}
variable "public_key" {}
variable "private_key" {}
```

4. Create outputs.tf with the following content:

```
output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}
```

- **Save screenshot as:** task1_outputs_tf.png – content of outputs.tf file.

A screenshot of a terminal window with the title bar "GNU nano 7.2 outputs.tf *". The terminal shows the content of a file named "outputs.tf" using the GNU nano 7.2 editor. The content of the file is a single Terraform output definition.

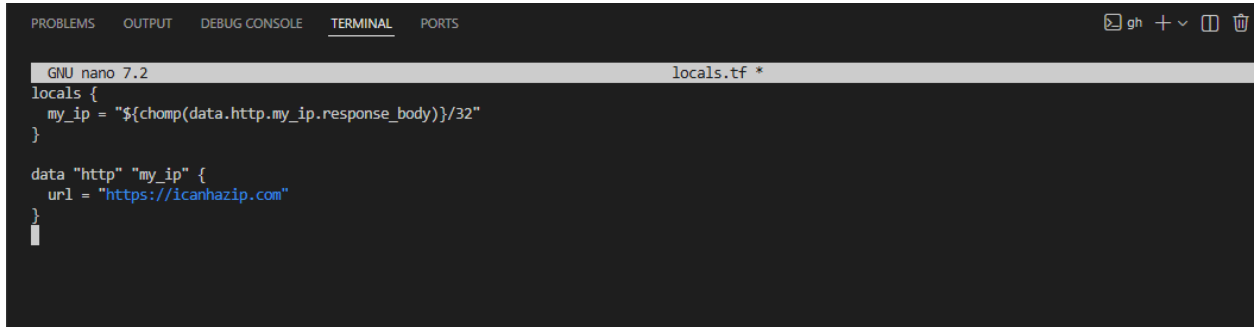
```
GNU nano 7.2 outputs.tf *
output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}
```

5. Create locals.tf with the following content:

```
locals {  
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"  
}
```

```
data "http" "my_ip" {  
  url = "https://icanhazip.com"  
}
```

- **Save screenshot as:** task1_locals_tf.png – content of locals.tf file.

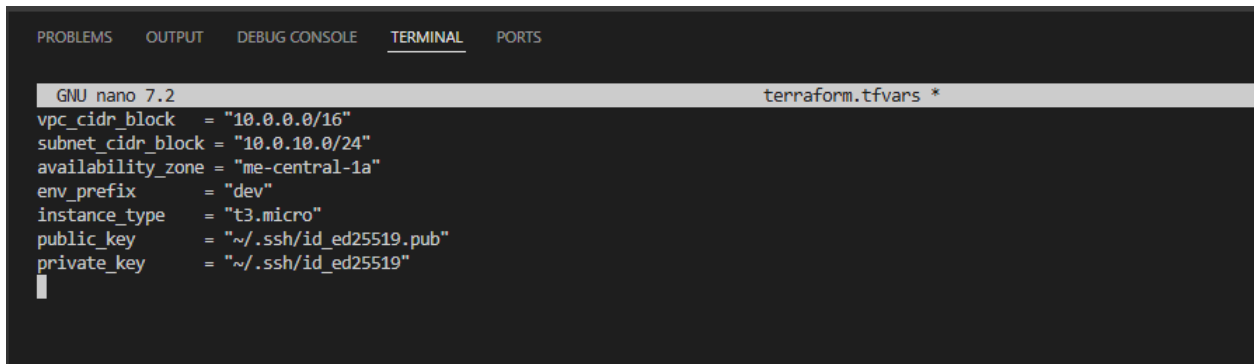
A screenshot of a terminal window with a dark background. The terminal shows the content of a file named 'locals.tf' being edited with 'GNU nano 7.2'. The code defines a local variable 'my_ip' based on an HTTP request to 'https://icanhazip.com' and a data source 'http' named 'my_ip' with the same URL.

```
GNU nano 7.2 locals.tf *  
locals {  
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"  
}  
  
data "http" "my_ip" {  
  url = "https://icanhazip.com"  
}
```

6. Create terraform.tfvars with the following content:

```
vpc_cidr_block = "10.0.0.0/16"  
subnet_cidr_block = "10.0.10.0/24"  
availability_zone = "me-central-1a"  
env_prefix = "dev"  
instance_type = "t3.micro"  
public_key = "~/.ssh/id_ed25519.pub"  
private_key = "~/.ssh/id_ed25519"
```

- **Save screenshot as:** task1_terraform_tfvars.png – content of terraform.tfvars file.

A screenshot of a terminal window with a dark background. The terminal shows the content of a file named 'terraform.tfvars' being edited with 'GNU nano 7.2'. The code defines several variables for AWS resources, including VPC CIDR block, subnet CIDR block, availability zone, environment prefix, instance type, and SSH keys.

```
GNU nano 7.2 terraform.tfvars *  
vpc_cidr_block = "10.0.0.0/16"  
subnet_cidr_block = "10.0.10.0/24"  
availability_zone = "me-central-1a"  
env_prefix = "dev"  
instance_type = "t3.micro"  
public_key = "~/.ssh/id_ed25519.pub"  
private_key = "~/.ssh/id_ed25519"
```

7. Create main.tf with the following content:

```
provider "aws" {  
  shared_config_files = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}
```

```
resource "aws_vpc" "myapp_vpc" {  
  cidr_block = var.vpc_cidr_block  
  tags = {  
    Name = "${var.env_prefix}-vpc"  
  }  
}
```

```

resource "aws_subnet" "myapp_subnet_1" {
  vpc_id      = aws_vpc.myapp_vpc.id
  cidr_block  = var.subnet_cidr_block
  availability_zone = var.availability_zone
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}

resource "aws_default_route_table" "main_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }
  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_default_security_group" "default_sg" {
  vpc_id      = aws_vpc.myapp_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [local.my_ip]
  }
  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    prefix_list_ids = []
  }
  tags = {
    Name = "${var.env_prefix}-default-sg"
  }
}

```

```

resource "aws_key_pair" "ssh-key" {
  key_name = "serverkey"
  public_key = file(var.public_key)
}

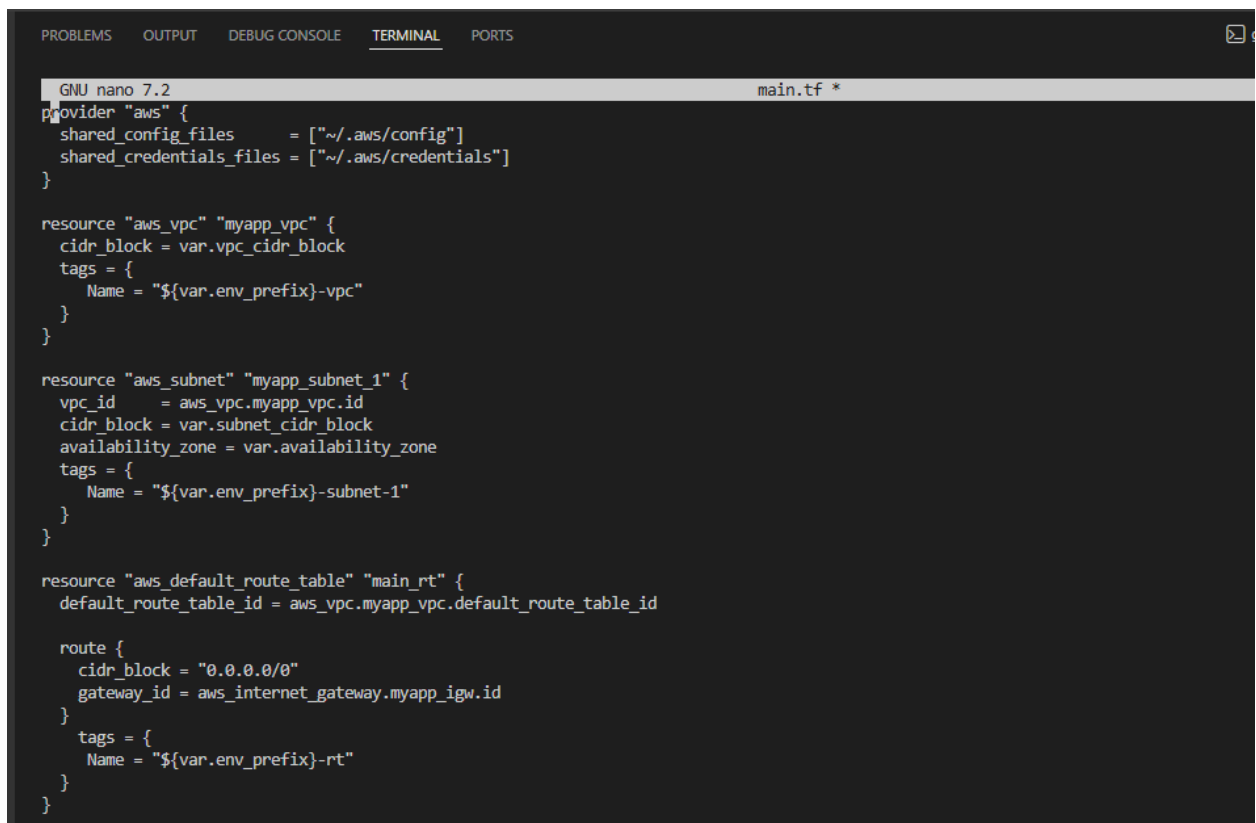
resource "aws_instance" "myapp-server" {
  ami          = "ami-05524d6658fcf35b6" # Amazon Linux 2023 Kernel 6.1 AMI
  instance_type = var.instance_type
  subnet_id    = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg. id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key. key_name

  user_data = file("./entry-script.sh")

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}

```

- **Save screenshot as:** task1_main_tf.png – content of main.tf file.



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
GNU nano 7.2 main.tf *
provider "aws" {
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

resource "aws_subnet" "myapp_subnet_1" {
  vpc_id = aws_vpc.myapp_vpc.id
  cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}

resource "aws_default_route_table" "main_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }
  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

```

8. Create entry-script.sh with the following content:

```

#!/bin/bash
set -e
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx

```

- Save screenshot as: task1_entry_script.png – content of entry-script.sh file.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

GNU nano 7.2                                     entry-script.sh *
#!/bin/bash
set -e
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx

```

9. Generate SSH key pair if not already exists:
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""

- Save screenshot as: task1_ssh_keygen.png – terminal showing SSH key generation.

```

@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /home/codespace/.ssh/id_ed25519
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:unKdmfWoCpXa7/Ma9muHzZmtZNZJmGR8+cI9ESaekoo codespace@codespaces-408b64
The key's randomart image is:
+--[ED25519 256]--+
|      . . . . .|
|      .O.=.    |
|      oo+ O.   |
|      .. .O.O.O|
|      oES. o ...|
|      + . . O .|
|      o O.O= *++O|
|      ...+*O++B .|
|      ooo=*+o.. |
+----[SHA256]-----+
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $

```

10. Initialize Terraform:
terraform init

- Save screenshot as: task1_terraform_init.png – terraform init output.

```

@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/http...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
- Installing hashicorp/aws v6.27.0...
- Installed hashicorp/aws v6.27.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $

```

11. Apply the configuration:

- terraform apply -auto-approve
- **Save screenshot as:** task1_terraform_apply.png – terraform apply output showing resources created.

```
Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + aws_instance_public_ip = (known after apply)
aws_key_pair.ssh-key: Creating...
aws_vpc.myapp_vpc: Creating...
aws_key_pair.ssh-key: Creation complete after 0s [id=serverkey]
aws_vpc.myapp_vpc: Creation complete after 1s [id=vpc-0e8da6c18d6f07162]
aws_internet_gateway.myapp_igw: Creating...
aws_subnet.myapp_subnet_1: Creating...
aws_default_security_group.default_sg: Creating...
aws_internet_gateway.myapp_igw: Creation complete after 0s [id=igw-0ffb74f27a99821c3]
aws_default_route_table.main_rt: Creating...
aws_subnet.myapp_subnet_1: Creation complete after 1s [id=subnet-02acdb9d621d3c685]
aws_default_route_table.main_rt: Creation complete after 1s [id=rtb-08966ba3dcb7d1e0d]
aws_default_security_group.default_sg: Creation complete after 3s [id=sg-03c17e26013486866]
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 12s [id=i-05de7e15e876810ec]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "51.112.180.16"
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

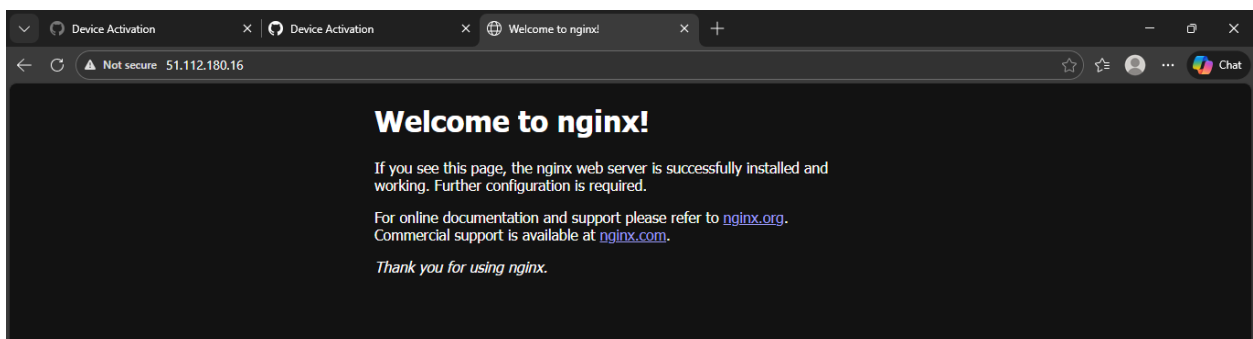
12. Display the output:
terraform output

- **Save screenshot as:** task1_terraform_output.png – terraform output showing public IP.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform output
aws_instance_public_ip = "51.112.180.16"
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

13. Test nginx in browser:

- Open browser and navigate to http://<public-ip>
- **Save screenshot as:** task1_nginx_browser.png – browser showing nginx default page.



14. Destroy resources:
terraform destroy

- Type yes when prompted for confirmation.
- **Save screenshot as:** task1_terraform_destroy.png – terraform destroy output.


```
Enter a value: yes

aws_default_route_table.main_rt: Destroying... [id=rtb-08966ba3dcb7d1e0d]
aws_default_route_table.main_rt: Destruction complete after 0s
aws_instance.myapp-server: Destroying... [id=i-05de7e15e876810ec]
aws_internet_gateway.myapp_igw: Destroying... [id=igw-0ffb74f27a99821c3]
aws_instance.myapp-server: Still destroying... [id=i-05de7e15e876810ec, 00m10s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0ffb74f27a99821c3, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-05de7e15e876810ec, 00m20s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0ffb74f27a99821c3, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-05de7e15e876810ec, 00m30s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0ffb74f27a99821c3, 00m30s elapsed]
aws_internet_gateway.myapp_igw: Destruction complete after 37s
aws_instance.myapp-server: Still destroying... [id=i-05de7e15e876810ec, 00m40s elapsed]
aws_instance.myapp-server: Destruction complete after 40s
aws_subnet.myapp_subnet_1: Destroying... [id=subnet-02acdb9d621d3c685]
aws_key_pair.ssh-key: Destroying... [id=serverkey]
aws_default_security_group.default_sg: Destroying... [id=sg-03c17e26013486866]
aws_default_security_group.default_sg: Destruction complete after 0s
aws_key_pair.ssh-key: Destruction complete after 0s
aws_subnet.myapp_subnet_1: Destruction complete after 0s
aws_vpc.myapp_vpc: Destroying... [id=vpc-0e8da6c18d6f07162]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
@Hamail-maker →/workspaces/CC Hamail-Fatima 2023-BSE-023 Lab12 (main) $
```

Task 2 – Use remote-exec provisioner

In this task, you will replace the user_data approach with the remote-exec provisioner to install and configure nginx.

1. Modify the aws_instance resource in main.tf to use remote-exec provisioner:

Replace the user_data line with the following provisioner block:

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658f35b6"
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name       = aws_key_pair.ssh-key.key_name

  connection {
    type      = "ssh"
    user      = "ec2-user"
    private_key = file(var.private_key)
    host      = self.public_ip
  }

  provisioner "remote-exec" {
    inline = [
      "sudo yum update -y",
      "sudo yum install -y nginx",
      "sudo systemctl start nginx",
      "sudo systemctl enable nginx"
    ]
  }

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
```

- **Save screenshot as:** task2_main_tf_remote_exec.png – main.tf showing remote-exec provisioner.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
GNU nano 7.2 main.tf
}
}
resource "aws_key_pair" "ssh-key" {
  key_name = "serverkey"
  public_key = file(var.public_key)
}

resource "aws_instance" "myapp-server" {
  ami = "ami-05524d6658fcf35b6" # Amazon Linux 2023 Kernel 6.1 AMI
resource "aws_instance" "myapp-server" {
  ami = "ami-05524d6658fcf35b6" # Amazon Linux 2023 Kernel 6.1 AMI
  instance_type = var.instance_type
  subnet_id = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key.key_name

  connection {
    type = "ssh"
    user = "ec2-user"
    private_key = file(var.private_key)
    host = self.public_ip
  }

  provisioner "remote-exec" {
    inline = [
      "sudo yum update -y",
      "sudo yum install -y nginx",
      "sudo systemctl start nginx",
    ]
  }
}
```

2. Apply the configuration:

terraform apply -auto-approve

- **Save screenshot as:** task2_terraform_apply.png – terraform apply output showing remote-exec execution.

```
aws_instance.myapp-server (remote-exec): Installing : gener [==== ] 6/7
aws_instance.myapp-server (remote-exec): Installing : gener [===== ] 6/7
aws_instance.myapp-server (remote-exec): Installing : generic-logo 6/7
aws_instance.myapp-server (remote-exec): Installing : nginx [ ] 7/7
aws_instance.myapp-server (remote-exec): Installing : nginx [== ] 7/7
aws_instance.myapp-server (remote-exec): Installing : nginx [=== ] 7/7
aws_instance.myapp-server (remote-exec): Installing : nginx [===== ] 7/7
aws_instance.myapp-server (remote-exec): Installing : nginx-1:1.28 7/7
aws_instance.myapp-server (remote-exec): Running scriptlet: nginx-1:1.28 7/7
aws_instance.myapp-server (remote-exec): Verifying : generic-logo 1/7
aws_instance.myapp-server (remote-exec): Verifying : gperf-tools-1 2/7
aws_instance.myapp-server (remote-exec): Verifying : libunwind-1. 3/7
aws_instance.myapp-server (remote-exec): Verifying : nginx-1:1.28 4/7
aws_instance.myapp-server (remote-exec): Verifying : nginx-core-1 5/7
aws_instance.myapp-server (remote-exec): Verifying : nginx-filesy 6/7
aws_instance.myapp-server (remote-exec): Verifying : nginx-mimety 7/7

aws_instance.myapp-server (remote-exec): Installed:
aws_instance.myapp-server (remote-exec): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
aws_instance.myapp-server (remote-exec): gperf-tools-libs-2.9.1-1.amzn2023.0.3.x86_64
aws_instance.myapp-server (remote-exec): libunwind-1.4.0-5.amzn2023.0.3.x86_64
aws_instance.myapp-server (remote-exec): nginx-1:1.28.0-1.amzn2023.0.2.x86_64
aws_instance.myapp-server (remote-exec): nginx-core-1:1.28.0-1.amzn2023.0.2.x86_64
aws_instance.myapp-server (remote-exec): nginx-filessystem-1:1.28.0-1.amzn2023.0.2.noarch
aws_instance.myapp-server (remote-exec): nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

aws_instance.myapp-server (remote-exec): Complete!
aws_instance.myapp-server (remote-exec): Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr
aws_instance.myapp-server: Creation complete after 1m3s [id=i-0046125153fb03672]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "3.29.232.58"

@remotell-makel 77/WORKSPACES/CC namall-fallm 2023-03-02 14:11 (md11) P
```

3. Display the output:

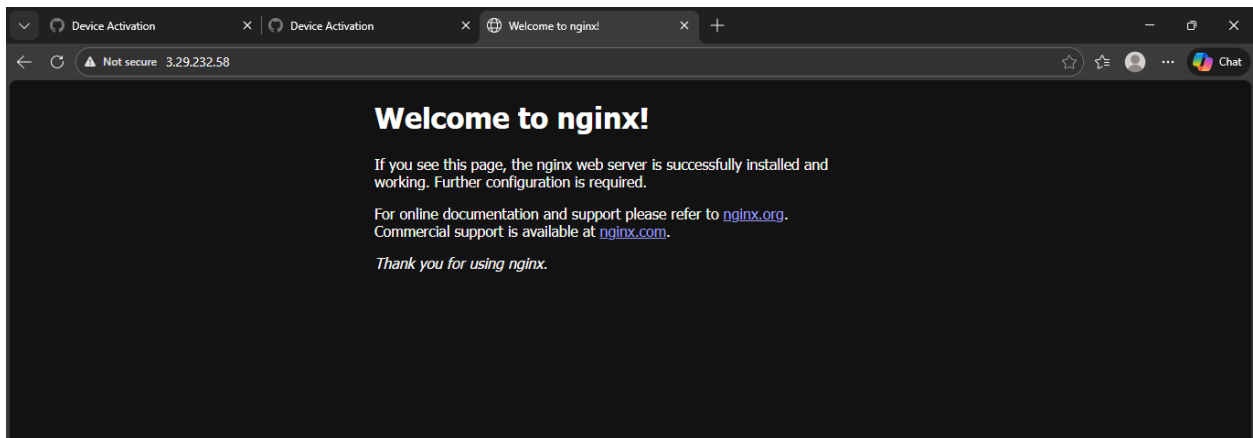
terraform output

- **Save screenshot as:** task2_terraform_output.png — terraform output showing public IP.

```
@Hamail-maker → /workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform output
aws_instance_public_ip = "3.29.232.58"
@Hamail-maker → /workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

4. Test nginx in browser:

- Open browser and navigate to `http://<public-ip>`
- **Save screenshot as:** task2_nginx_browser.png — browser showing nginx default page.



Task 3 — Use file and local-exec provisioners

In this task, you will add the file provisioner to upload the script and the local-exec provisioner to log instance information locally.

1. Modify the aws_instance resource in main.tf to include all three provisioners:

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6"
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name       = aws_key_pair.ssh-key.key_name

  connection {
    type     = "ssh"
    user     = "ec2-user"
    private_key = file(var.private_key)
    host     = self.public_ip
  }

  provisioner "file" {
    source = "./entry-script.sh"
    destination = "/home/ec2-user/entry-script-on-ec2.sh"
  }

  provisioner "remote-exec" {
    inline = [
      sudo chmod +x /home/ec2-user/entry-script-on-ec2.sh ,
```

```

    "sudo /home/ec2-user/entry-script-on-ec2.sh"
  ]
}

provisioner "local-exec" {
  command = <<-EOF
    echo Instance ${self.id} with public IP ${self.public_ip} has been created
  EOF
}

tags = {
  Name = "${var.env_prefix}-ec2-instance"
}
}

```

- Save screenshot as: task3_main_tf_all_provisioners.png – main.tf showing file, remote-exec, and local-exec provisioners.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
GNU nano 7.2  main.tf

resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6"
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name       = aws_key_pair.ssh-key.key_name

  connection {
    type = "ssh"
    user = "ec2-user"
    private_key = file(var.private_key)
    host = self.public_ip
  }

  # 1 FILE PROVISIONER
  provisioner "file" {
    source = "./entry-script.sh"
    destination = "/home/ec2-user/entry-script-on-ec2.sh"
  }

  # 2 REMOTE-EXEC PROVISIONER
  provisioner "remote-exec" {
    inline = [
      "sudo chmod +x /home/ec2-user/entry-script-on-ec2.sh",
      "sudo /home/ec2-user/entry-script-on-ec2.sh"
    ]
  }

  # 3 LOCAL-EXEC PROVISIONER

```

2. Apply the configuration:
 terraform apply -auto-approve

- Save screenshot as: task3_terraform_apply.png – terraform apply output showing all provisioners execution.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_default_route_table.main_rt will be created
+ resource "aws_default_route_table" "main_rt" {
  + arn                = (known after apply)
  + default_route_table_id = (known after apply)
  + id                 = (known after apply)
  + owner_id           = (known after apply)
  + region              = "me-central-1"
  + route               = [
    + {
      + cidr_block      = "0.0.0.0/0"
      + gateway_id       = (known after apply)
      # (10 unchanged attributes hidden)
    },
  ]
  + tags               = {
    + "Name" = "dev-rt"
  }
  + tags_all           = {
    + "Name" = "dev-rt"
  }
  + vpc_id              = (known after apply)
}
```

3. Display the output:

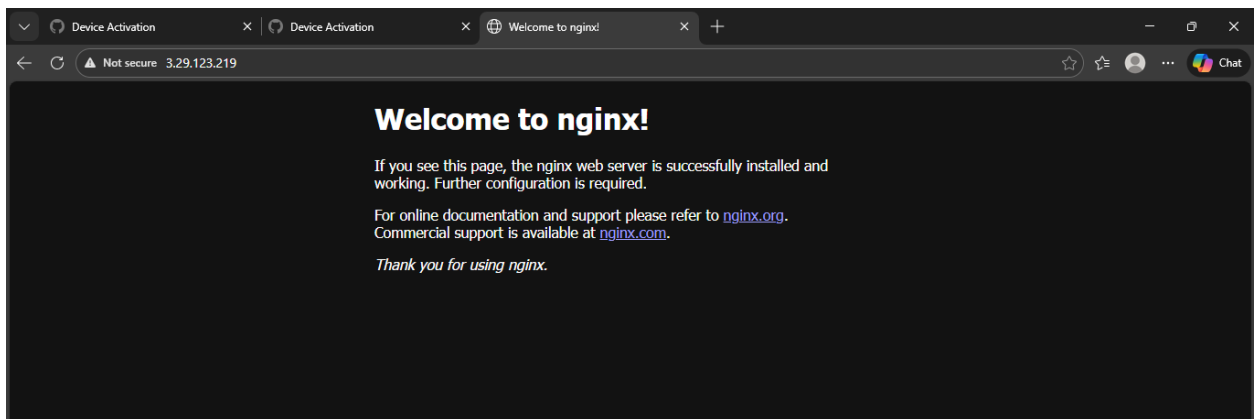
terraform output

- Save screenshot as: task3_terraform_output.png – terraform output showing public IP.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform output
aws_instance_public_ip = "3.29.123.219"
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

4. Test nginx in browser:

- Open browser and navigate to `http://<public-ip>`
- Save screenshot as: task3_nginx_browser.png – browser showing nginx default page.



5. Destroy the resources:

terraform destroy

- Type yes when prompted.
- Save screenshot as: task3_terraform_destroy.png – terraform destroy output.

```

- aws_instance_public_ip = "3.29.123.219" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_default_route_table.main_rt: Destroying... [id=rtb-0dcbed0bfa44890ba]
aws_instance.myapp-server: Destroying... [id=i-04510e5a18cc94cc5]
aws_default_route_table.main_rt: Destruction complete after 0s
aws_internet_gateway.myapp_igw: Destroying... [id=igw-0126a3608bace919e]
aws_instance.myapp-server: Still destroying... [id=i-04510e5a18cc94cc5, 00m10s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0126a3608bace919e, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-04510e5a18cc94cc5, 00m20s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0126a3608bace919e, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-04510e5a18cc94cc5, 00m30s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0126a3608bace919e, 00m30s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-04510e5a18cc94cc5, 00m40s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0126a3608bace919e, 00m40s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-04510e5a18cc94cc5, 00m50s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0126a3608bace919e, 00m50s elapsed]
aws_internet_gateway.myapp_igw: Destruction complete after 58s
aws_instance.myapp-server: Still destroying... [id=i-04510e5a18cc94cc5, 01m00s elapsed]
aws_instance.myapp-server: Destruction complete after 1m1s
aws_key_pair.ssh-key: Destroying... [id=serverkey]
aws_subnet.myapp_subnet_1: Destroying... [id=subnet-0b88db7bec382fa00]
aws_default_security_group.default_sg: Destroying... [id=sg-0385af99c07dcaf24]
aws_default_security_group.default_sg: Destruction complete after 0s
aws_key_pair.ssh-key: Destruction complete after 0s
aws_subnet.myapp_subnet_1: Destruction complete after 0s
aws_vpc.myapp_vpc: Destroying... [id=vpc-03ad71d6c21e42be8]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.

```

6. Remove the provisioners and restore user_data:

Replace the connection and provisioner blocks with:

user_data = file("./entry-script.sh")

- Save screenshot as: task3_main_tf_restored.png – main.tf showing user_data restored.

```

resource "aws_instance" "myapp-server" {
  ami                = "ami-05524d6658fcf35b6"
  instance_type      = var.instance_type
  subnet_id          = aws_subnet.myapp_subnet_1.id
  security_groups    = [aws_default_security_group.default_sg.id]
  availability_zone   = var.availability_zone
  associate_public_ip_address = true
  key_name            = aws_key_pair.ssh-key.key_name

  user_data = file("./entry-script.sh")

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}

```

Task 4 – Create Terraform modules (subnet module)

In this task, you will create a reusable subnet module to organize your infrastructure code better.

1. Create the module directory structure:

mkdir -p modules/subnet

touch modules/subnet/main.tf

touch modules/subnet/variables.tf

touch modules/subnet/outputs.tf

- **Save screenshot as:** task4_module_structure.png – terminal showing module directory structure (use tree or ls -R).

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ nano main.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ mkdir -p modules/subnet
touch modules/subnet/main.tf
touch modules/subnet/variables.tf
touch modules/subnet/outputs.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ ls -R modules
modules:
subnet
modules/subnet:
main.tf outputs.tf variables.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

2. Create modules/subnet/variables.tf:

```
variable "vpc_id" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}
variable "default_route_table_id" {}
```

- **Save screenshot as:** task4_subnet_variables.png – content of modules/subnet/variables.tf.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ nano modules/subnet/variables.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat modules/subnet/variables.tf
variable "vpc_id" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}
variable "default_route_table_id" {}
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

3. Create modules/subnet/main.tf:

```
resource "aws_subnet" "myapp_subnet_1" {
  vpc_id      = var.vpc_id
  cidr_block  = var.subnet_cidr_block
  availability_zone = var.availability_zone
  map_public_ip_on_launch = true
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}

resource "aws_default_route_table" "main_rt" {
  default_route_table_id = var.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }
  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = var.vpc_id
  tags = {
    Name = "${var.env_prefix}-igw"
  }
}
```

- **Save screenshot as:** task4_subnet_main.png – content of modules/subnet/main.tf.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ nano modules/subnet/main.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat modules/subnet/main.tf
resource "aws_subnet" "myapp_subnet_1" {
  vpc_id            = var.vpc_id
  cidr_block        = var.subnet_cidr_block
  availability_zone  = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}

resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = var.vpc_id

  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_default_route_table" "main_rt" {
  default_route_table_id = var.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

4. Create modules/subnet/outputs.tf:

```
output "subnet" {
  value = aws_subnet.myapp_subnet_1
}
```

- **Save screenshot as:** task4_subnet_outputs.png – content of modules/subnet/outputs.tf.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ nano modules/subnet/outputs.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat modules/subnet/outputs.tf
output "subnet" {
  value = aws_subnet.myapp_subnet_1
}
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

5. Modify the root main.tf to use the subnet module:

Remove the subnet, route table, and internet gateway resources and replace them with:

```
module "myapp-subnet" {
  source = "../modules/subnet"
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  env_prefix = var.env_prefix
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
}
```

And update the instance resource to reference the module output:

```
resource "aws_instance" "myapp-server" {
  # ... other settings ...
  subnet_id = module.myapp-subnet.subnet.id
  # ... rest of configuration ...
}
```

- **Save screenshot as:** task4_main_tf_with_module.png – main.tf showing module usage.


```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat main.tf
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

module "myapp-subnet" {
  source = "../modules/subnet"

  vpc_id            = aws_vpc.myapp_vpc.id
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone  = var.availability_zone
  env_prefix         = var.env_prefix
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
}

resource "aws_default_security_group" "default_sg" {
  vpc_id = aws_vpc.myapp_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [local.my_ip]
  }

  ingress {
    from_port = 80
  }
}
```

6. Initialize Terraform to download the module:

terraform init

- **Save screenshot as:** task4_terraform_init.png – terraform init output showing module initialization.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform init
Initializing the backend...
Initializing modules...
- myapp-subnet in modules/subnet
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/http from the dependency lock file
- Using previously-installed hashicorp/aws v6.27.0
- Using previously-installed hashicorp/http v3.5.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

7. Apply the configuration:

terraform apply -auto-approve

- **Save screenshot as:** task4_terraform_apply.png – terraform apply output with module.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_default_security_group.default_sg will be created
+ resource "aws_default_security_group" "default_sg" {
+   arn                = (known after apply)
+   description        = (known after apply)
+   egress              = [
+     {
+       cidr_blocks      = [
+         "0.0.0.0/0",
+       ]
+       from_port         = 0
+       ipv6_cidr_blocks = []
+       prefix_list_ids   = []
+       protocol          = "-1"
+       security_groups   = []
+       self              = false
+       to_port           = 0
+     }
+   ]
+   # 41 unchanged attributes hidden
}
```

8. Display the output:

terraform output

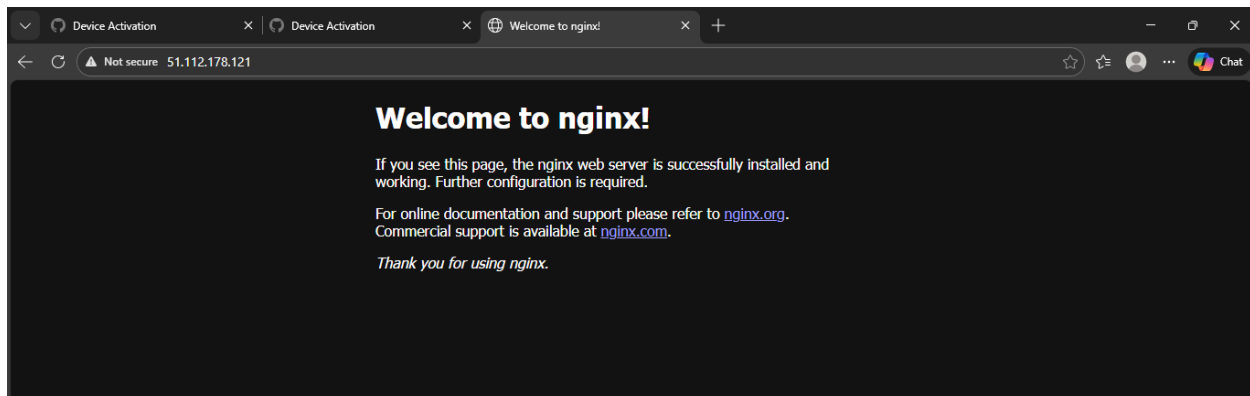
- **Save screenshot as:** task4_terraform_output.png – terraform output showing public IP.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform output
aws_instance_public_ip = "51.112.178.121"
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

9. Test nginx in browser:

- Open browser and navigate to `http://<public-ip>`

Save screenshot as: task4_nginx_browser.png – browser showing nginx default page.



Task 5 – Create webserver module

In this task, you will create a reusable webserver module for EC2 instances.

1. Create the webserver module directory structure:

```
mkdir -p modules/webserver
```

```
touch modules/webserver/main.tf
```

```
touch modules/webserver/variables.tf
```

```
touch modules/webserver/outputs.tf
```

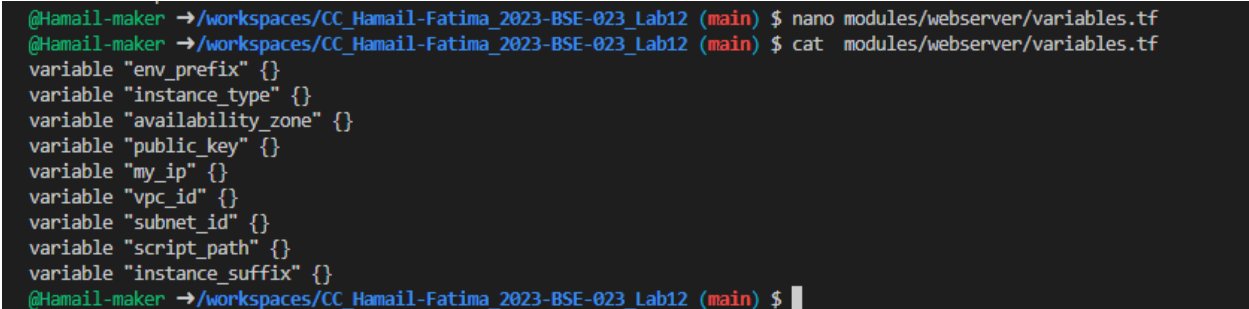
- **Save screenshot as:** task5_webserver_module_structure.png – terminal showing webserver module directory.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ mkdir -p modules/webserver
touch modules/webserver/main.tf
touch modules/webserver/variables.tf
touch modules/webserver/outputs.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ ls -R modules/webserver
modules/webserver:
main.tf  outputs.tf  variables.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

2. Create modules/webserver/variables.tf:

```
variable "env_prefix" {}
variable "instance_type" {}
variable "availability_zone" {}
variable "public_key" {}
variable "my_ip" {}
variable "vpc_id" {}
variable "subnet_id" {}
variable "script_path" {}
variable "instance_suffix" {}
```

- **Save screenshot as:** task5_webserver_variables.png – content of modules/webserver/variables. tf.



```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ nano modules/webserver/variables.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat modules/webserver/variables.tf
variable "env_prefix" {}
variable "instance_type" {}
variable "availability_zone" {}
variable "public_key" {}
variable "my_ip" {}
variable "vpc_id" {}
variable "subnet_id" {}
variable "script_path" {}
variable "instance_suffix" {}
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

3. Create modules/webserver/main.tf:

```
resource "aws_security_group" "web_sg" {
  vpc_id    = var.vpc_id
  name      = "${var.env_prefix}-web-sg-${var.instance_suffix}"
  description = "Security group for web server allowing HTTP, HTTPS and SSH"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
  }
  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    prefix_list_ids = []
  }
  tags = {
```

```

    Name = "${var.env_prefix}-default-sg"
  }
}

resource "aws_key_pair" "ssh-key" {
  key_name = "${var.env_prefix}-serverkey-${var.instance_suffix}"
  public_key = file(var.public_key)
}

resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023 Kernel 6.1 AMI
  instance_type = var.instance_type
  subnet_id     = var.subnet_id
  vpc_security_group_ids = [aws_security_group.web_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key.key_name

  user_data = file(var.script_path)

  tags = {
    Name = "${var.env_prefix}-ec2-instance-${var.instance_suffix}"
  }
}

```

- **Save screenshot as:** task5_webserver_main.png – content of modules/webserver/main.tf.

```

@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat modules/webserver/main.tf
resource "aws_security_group" "web_sg" {
  vpc_id     = var.vpc_id
  name       = "${var.env_prefix}-web-sg-${var.instance_suffix}"
  description = "Security group for web server allowing HTTP, HTTPS and SSH"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    prefix_list_ids = []
  }

  tags = {

```

4. Create modules/webserver/outputs.tf:

```

output "aws_instance" {
  value = aws_instance.myapp-server
}

```

- **Save screenshot as:** task5_webserver_outputs.png – content of modules/webserver/outputs.tf.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat modules/webserver/outputs.tf
output "aws_instance" {
  value = aws_instance.myapp-server
}
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

5. Modify the root main.tf:

Remove the security group, key pair, and instance resources. Replace them with:

```
module "myapp-webserver" {
  source = "../modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_id = module.myapp-subnet.subnet.id
  script_path = "../entry-script.sh"
  instance_suffix = "0"
}
```

- **Save screenshot as:** task5_main_tf_webserver_module.png – main.tf showing webserver module usage.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat main.tf
provider "aws" {
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block

  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

module "myapp-subnet" {
  source = "../modules/subnet"

  vpc_id = aws_vpc.myapp_vpc.id
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  env_prefix = var.env_prefix
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
}

module "myapp-webserver" {
  source = "../modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_id = module.myapp-subnet.subnet.id
  script_path = "../entry-script.sh"
  instance_suffix = "0"
}
```

6. Update outputs.tf:

```
output "webserver_public_ip" {
  value = module.myapp-webserver.aws_instance.public_ip
}
```

- Save screenshot as: task5_outputs_updated.png — updated outputs.tf.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat outputs.tf
output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}
output "webserver_public_ip" {
  value = module.myapp-webserver.aws_instance.public_ip
}
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

7. Initialize Terraform:

terraform init

- Save screenshot as: task5_terraform_init.png — terraform init output.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform init
Initializing the backend...
Initializing modules...
- myapp-webserver in modules/webserver
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/http from the dependency lock file
- Using previously-installed hashicorp/aws v6.27.0
- Using previously-installed hashicorp/http v3.5.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

8. Apply the configuration:

terraform apply -auto-approve

- Save screenshot as: task5_terraform_apply.png — terraform apply output with webserver module.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
aws_key_pair.ssh-key: Refreshing state... [id=serverkey]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0e326cecedd45fae1]
aws_default_security_group.default_sg: Refreshing state... [id=sg-0398227fca6dd8aff]
aws_instance.myapp-server: Refreshing state... [id=i-0ae8941c7d3c27edd]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0d0d6a24d0537ada0]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0f5fe1fee2b15f4ad]
module.myapp-subnet.aws_default_route_table.main_rt: Refreshing state... [id=rtb-00bfc023ad38ba8d]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

# aws_default_security_group.default_sg will be destroyed
# (because aws_default_security_group.default_sg is not in configuration)
- resource "aws_default_security_group" "default_sg" {
  - arn              = "arn:aws:ec2:me-central-1:880794402980:security-group/sg-0398227fca6dd8aff" -> null
  - description     = "default VPC security group" -> null
  - egress          = [

```

9. Display the output:

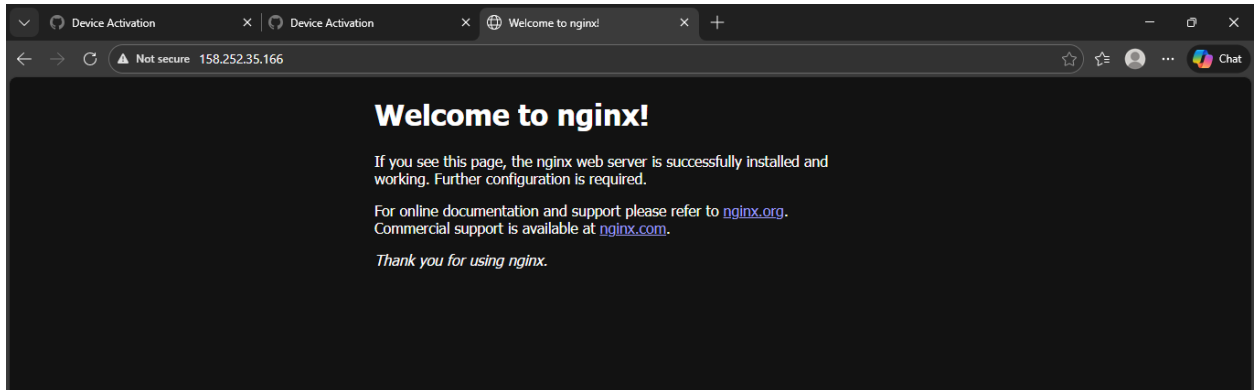
terraform output

- Save screenshot as: task5_terraform_output.png — terraform output showing webserver public IP.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform output
webserver_public_ip = "158.252.35.166"
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

10. Test nginx in browser:

- Open browser and navigate to `http://<public-ip>`
- **Save screenshot as:** `task5_nginx_browser.png` – browser showing nginx default page.



11. Destroy resources:

terraform destroy

- Type yes when prompted.
- **Save screenshot as:** `task5_terraform_destroy.png` – terraform destroy output.

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

module.myapp-subnet.aws_default_route_table.main_rt: Destroying... [id=rtb-00bfdc023ad38ba8d]
module.myapp-subnet.aws_default_route_table.main_rt: Destruction complete after 0s
module.myapp-webserver.aws_instance.myapp-server: Destroying... [id=i-0ca889d55276b2f4e]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Destroying... [id=igw-0d0d6a24d0537ada0]
module.myapp-webserver.aws_instance.myapp-server: Still destroying... [id=i-0ca889d55276b2f4e, 00m10s elapsed]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0d0d6a24d0537ada0, 00m10s elapsed]
module.myapp-webserver.aws_instance.myapp-server: Still destroying... [id=i-0ca889d55276b2f4e, 00m20s elapsed]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0d0d6a24d0537ada0, 00m20s elapsed]
module.myapp-webserver.aws_instance.myapp-server: Still destroying... [id=i-0ca889d55276b2f4e, 00m30s elapsed]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0d0d6a24d0537ada0, 00m30s elapsed]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Destruction complete after 37s
module.myapp-webserver.aws_instance.myapp-server: Still destroying... [id=i-0ca889d55276b2f4e, 00m40s elapsed]
module.myapp-webserver.aws_instance.myapp-server: Still destroying... [id=i-0ca889d55276b2f4e, 00m50s elapsed]
module.myapp-webserver.aws_instance.myapp-server: Destruction complete after 50s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destroying... [id=subnet-0f5fe1fee2b15f4ad]
module.myapp-webserver.aws_key_pair.ssh-key: Destroying... [id=dev-serverkey-0]
module.myapp-webserver.aws_security_group.web_sg: Destroying... [id=sg-0a1ec43bf9c611461]
module.myapp-webserver.aws_key_pair.ssh-key: Destruction complete after 1s
module.myapp-webserver.aws_security_group.web_sg: Destruction complete after 1s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-0e326cecedd45fae1]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
@Hamail-maker →/workspaces/CC Hamail-Fatima 2023-BSE-023 Lab12 (main) $
```

Task 6 – Configure HTTPS with self-signed certificates

In this task, you will configure Nginx to serve traffic over HTTPS using self-signed certificates.

1. Update entry-script.sh with SSL configuration:

```
#!/bin/bash
set -e
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx
```

```

# Create directories for SSL certificates if they don't exist
mkdir -p /etc/ssl/private
mkdir -p /etc/ssl/certs

# Get IMDSv2 token
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Get current public IP
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4)

PUBLIC_HOSTNAME=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-hostname)

# Generate self-signed certificate with dynamic IP
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/CN=$PUBLIC_IP" \
-addext "subjectAltName=IP:$PUBLIC_IP" \
-addext "basicConstraints=CA:FALSE" \
-addext "keyUsage=digitalSignature,keyEncipherment" \
-addext "extendedKeyUsage=serverAuth"

echo "Self-signed certificate created for IP: $PUBLIC_IP"

# Backup existing nginx. conf
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak

# Overwrite nginx.conf with the desired content
cat <<EOF > /etc/nginx/nginx.conf
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx. pid;

events {
    worker_connections 1024;
}

http {
    log_format main "\$remote_addr - \$remote_user [\$time_local] \"\$request\"
        \"\$status \$body_bytes_sent \"\$http_referer\"
        \"\$http_user_agent\" \"\$http_x_forwarded_for\";

    access_log /var/log/nginx/access.log main;

    sendfile          on;
    tcp_nopush        on;
    keepalive_timeout 65;
    types_hash_max_size 4096;

    include            /etc/nginx/mime.types;
    default_type       application/octet-stream;

```



```

upstream backend_servers {
    server 158.252.94.241:80;
    server 158.252.94.242:80 backup;
}

server {
    listen 443 ssl;
    server_name $PUBLIC_IP;
    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        # proxy_pass http://158.252.94.241:80;
        # proxy_pass http://backend_servers;

    }
}

server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}
}
EOF

```

Test and restart Nginx

systemctl restart nginx

- **Save screenshot as:** task6_entry_script_https.png – updated entry-script.sh with HTTPS configuration.

```

GNU nano 7.2                                     entry-script.sh *
#!/bin/bash
set -e
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx
mkdir -p /etc/ssl/private
mkdir -p /etc/ssl/certs
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4)

PUBLIC_HOSTNAME=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-hostname)
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/CN=$PUBLIC_IP" \
-addext "subjectAltName=IP:$PUBLIC_IP" \
-addext "basicConstraints=CA:FALSE" \
-addext "keyUsage=digitalSignature,keyEncipherment" \
-addext "extendedKeyUsage=serverAuth"
echo "Self-signed certificate created for IP: $PUBLIC_IP"
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
cat <<EOF > /etc/nginx/nginx.conf
# (Paste the full Nginx configuration from Task 6 here)
EOF
systemctl restart nginx

```

2. Apply the configuration:

terraform apply -auto-approve

- **Save screenshot as:** task6_terraform_apply.png – terraform apply output.

```
@Hamail-maker → /workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_vpc.myapp_vpc will be created
+ resource "aws_vpc" "myapp_vpc" {
  + arn                               = (known after apply)
  + cidr_block                       = "10.0.0.0/16"
  + default_network_acl_id          = (known after apply)
  + default_route_table_id         = (known after apply)
  + default_security_group_id      = (known after apply)
  + dhcp_options_id                = (known after apply)
  + enable_dns_hostnames            = (known after apply)
  + enable_dns_support              = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                             = (known after apply)
  + instance_tenancy                = "default"
  + ipv6_association_id            = (known after apply)
  + ipv6_cidr_block                 = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id            = (known after apply)
  + owner_id                       = (known after apply)
  + region                         = "me-central-1"
  + tags                           = {
    + "Name" = "dev-vpc"
  }
}
```

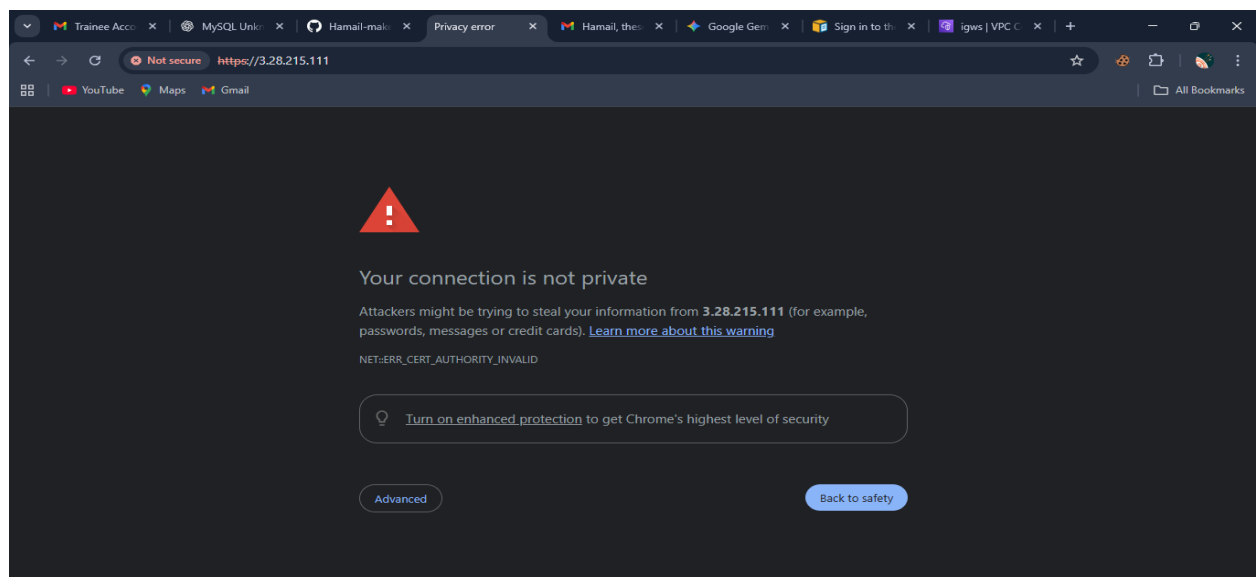
3. Display the output:
terraform output

- **Save screenshot as:** task6_terraform_output.png – terraform output showing public IP.

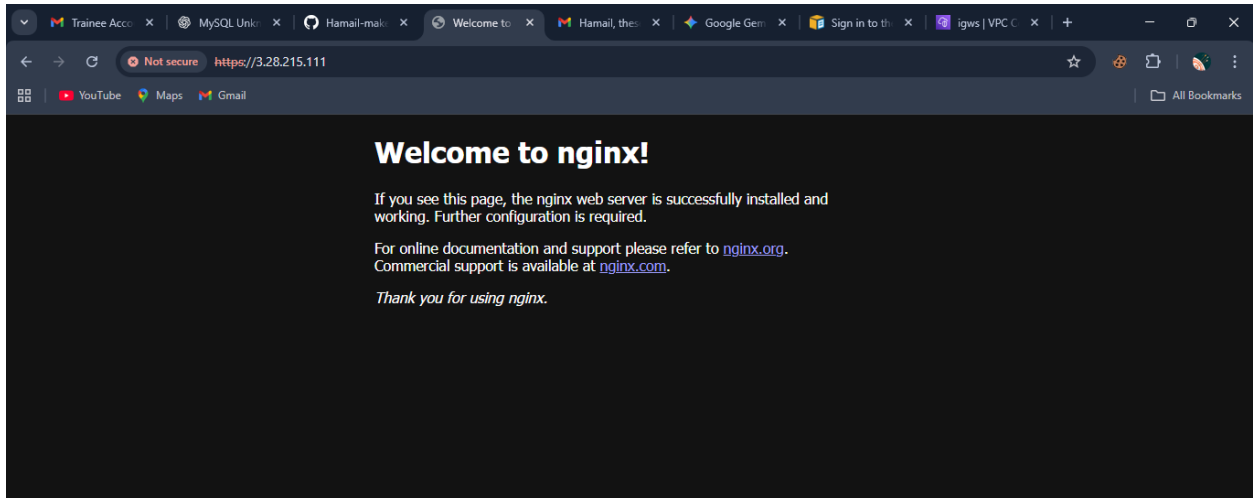
```
@Hamail-maker → /workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform output
webserver_public_ip = "3.28.137.138"
@Hamail-maker → /workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

4. Test HTTPS in browser:

- Open browser and navigate to https://<public-ip>
- You will see a warning: "Warning: Potential Security Risk Ahead"
- Click "Advanced" button
- Click "Accept the Risk and Continue"
- **Save screenshot as:** task6_browser_security_warning.png – browser showing security warning.

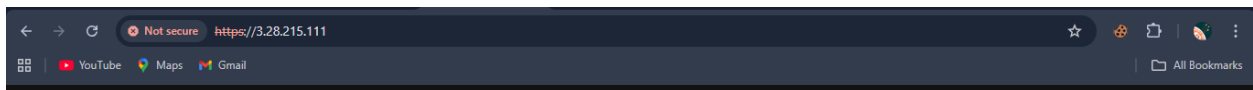


- **Save screenshot as:** task6_nginx_https_browser.png — browser showing nginx page over HTTPS after accepting risk.



5. Verify HTTP to HTTPS redirect:

- Open browser and navigate to `http://<public-ip>`
- Verify it redirects to `https://<public-ip>`
- **Save screenshot as:** task6_http_redirect.png — browser showing redirect from HTTP to HTTPS.



Task 7 — Configure Nginx as reverse proxy

In this task, you will create a backend web server and configure Nginx to act as a reverse proxy.

1. Create apache.sh script for backend web server:

```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
echo "<h1>Welcome to My Web Server</h1>" > /var/www/html/index.html
hostnamectl set-hostname myapp-webserver
echo "<h2>Hostname: $(hostname)</h2>" >> /var/www/html/index.html
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
echo "<h2>Private IP: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN"
http://169.254.169.254/latest/meta-data/local-ipv4)</h2>" >>
/var/www/html/index.html
echo "<h2>Public IP: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN"
http://169.254.169.254/latest/meta-data/public-ipv4)</h2>" >>
/var/www/html/index.html
echo "<h2>Public DNS: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN"
http://169.254.169.254/latest/meta-data/public-hostname)</h2>" >>
/var/www/html/index.html
echo "<h2>Deployed via Terraform</h2>" >> /var/www/html/index.html
```

- Save screenshot as: task7_apache_script.png – content of apache.sh file.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ nano apache.sh
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ chmod +x apache.sh
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat apache.sh
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd

# Create web page
echo "<h1>Welcome to My Web Server</h1>" > /var/www/html/index.html
hostnamectl set-hostname myapp-webserver
echo "<h2>Hostname: $(hostname)</h2>" >> /var/www/html/index.html

# Get EC2 metadata
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

echo "<h2>Private IP: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/local-ipv4)</h2>" >> /var/www/html/index.html
echo "<h2>Public IP: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/public-ipv4)</h2>" >> /var/www/html/index.html
echo "<h2>Public DNS: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/public-hostname)</h2>" >> /var/www/html/index.html
echo "<h2>Deployed via Terraform</h2>" >> /var/www/html/index.html
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

2. Add the backend web server module to main.tf:

```
module "myapp-web-1" {
  source = "./modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_id = module.myapp-subnet.subnet.id
  script_path = "./apache.sh"
  instance_suffix = "1"
}
```

- Save screenshot as: task7_main_tf_web1.png – main.tf showing myapp-web-1 module.

```
module "myapp-web-1" {
  source = "./modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_id = module.myapp-subnet.subnet.id
  script_path = "./apache.sh"
  instance_suffix = "1"
}
```

3. Update outputs.tf:

```
output "aws_web-1_public_ip" {
  value = module.myapp-web-1.aws_instance.public_ip
}
```

- Save screenshot as: task7_outputs_web1.png – outputs.tf with web-1 output.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ nano outputs.tf
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat outputs.tf

output "webserver_public_ip" {
  value = module.myapp-webserver.aws_instance.public_ip
}
output "aws_web-1_public_ip" {
  value = module.myapp-web-1.aws_instance.public_ip
}
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

4. Apply the configuration:

terraform apply -auto-approve

- Save screenshot as: task7_terraform_apply.png – terraform apply output showing both instances created.

```
# module.myapp-webserver.aws_key_pair.ssh-key will be destroyed
# (because aws_key_pair.ssh-key is not in configuration)
- resource "aws_key_pair" "ssh-key" {
  - arn          = "arn:aws:ec2:me-central-1:880794402980:key-pair/dev-key" -> null
  - fingerprint = "Gb2SYFzzpLBjyPa8ZfuILW6JBTIL0MSn9hY3+vffJw=" -> null
  - id          = "dev-key" -> null
  - key_name     = "dev-key" -> null
  - key_pair_id  = "key-09218c5dc08182ffc" -> null
  - key_type     = "ed25519" -> null
  - public_key   = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIG/6n6Q0QdiX9oIgPp1yKVC7pkQh86h90MsYz0iPf7B4M codespace@codespaces-408b64" -> null
  - region      = "me-central-1" -> null
  - tags        = {} -> null
  - tags_all    = {} -> null
  # (1 unchanged attribute hidden)
}

Plan: 2 to add, 0 to change, 1 to destroy.

Changes to Outputs:
+ aws_web-1_public_ip = (known after apply)
module.myapp-webserver.aws_key_pair.ssh-key: Destroying... [id=dev-key]
module.myapp-web-1.aws_instance.myapp-server: Creating...
module.myapp-web-1.aws_instance.web: Creating...
module.myapp-webserver.aws_key_pair.ssh-key: Destruction complete after 0s
module.myapp-web-1.aws_instance.web: Still creating... [00m10s elapsed]
module.myapp-web-1.aws_instance.myapp-server: Still creating... [00m10s elapsed]
module.myapp-web-1.aws_instance.myapp-server: Creation complete after 12s [id=i-0d22e5534e8a35bfa]
module.myapp-web-1.aws_instance.web: Creation complete after 12s [id=i-0d9734670eaeff1a4]

Apply complete! Resources: 2 added, 0 changed, 1 destroyed.

Outputs:

aws_web-1_public_ip = "40.172.177.11"
webserver_public_ip = "3.28.205.43"
```

5. Get the outputs:

terraform output

- Save screenshot as: task7_terraform_output.png – showing both public IPs.

```
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform output
aws_web-1_public_ip = "40.172.177.11"
webserver_public_ip = "3.28.205.43"
@Hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

6. SSH into the webserver (Nginx proxy server):

ssh ec2-user@<webserver-public-ip>

- Save screenshot as: task7_ssh_webserver.png – SSH session to webserver.

```
[Hmail-maker → /workspaces/CC Hmail-Fatima-2023-BSE-073_Lab12 (main)] $ ssh -i ~/.ssh/id_ed25519 ec2-user@3.28.205.43
```

The authenticity of host '3.28.205.43 (3.28.205.43)' can't be established.
ED25519 key fingerprint is SHA256:0a24CJbw+dQmZAZThkofB6vYqFRxTsATyWFKjVoeN+E.
This key is not known by any other names.

Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.28.205.43' (ED25519) to the list of known hosts.

```
y_##### Amazon Linux 2023\nnn_\####\n   _###|\n   \#/ https://aws.amazon.com/linux/amazon-linux-2023\n     Vw'->\n      ^--^-\n       | |\n       | |\n       | |\n    ___|_|___/\n    [ec2-user@ip-10-0-10-152 ~]$
```

```
[root@server1 ~]# cat /var/log/nginx/error.log
2026/01/03 19:22:54 [notice] 3116#3116: using the "epoll" event method
2026/01/03 19:22:54 [notice] 3116#3116: nginx/1.28.0
2026/01/03 19:22:54 [notice] 3116#3116: OS: Linux 6.1.158-180.294.amzn2023.x86_64
2026/01/03 19:22:54 [notice] 3116#3116: getrlimit(RLIMIT_NOFILE): 65535:65535
2026/01/03 19:22:54 [notice] 3158#3158: start worker processes
2026/01/03 19:22:54 [notice] 3158#3158: start worker process 3159
2026/01/03 19:22:54 [notice] 3158#3158: start worker process 3160
2026/01/03 19:22:55 [notice] 3158#3158: signal 3 (SIGQUIT) received from 1, shutting down
2026/01/03 19:22:55 [notice] 3159#3159: gracefully shutting down
2026/01/03 19:22:55 [notice] 3159#3159: exiting
2026/01/03 19:22:55 [notice] 3159#3159: exit
2026/01/03 19:22:55 [notice] 3160#3160: gracefully shutting down
2026/01/03 19:22:55 [notice] 3160#3160: exiting
2026/01/03 19:22:55 [notice] 3160#3160: exit
2026/01/03 19:22:55 [notice] 3158#3158: signal 17 (SIGCHLD) received from 3159
2026/01/03 19:22:55 [notice] 3158#3158: worker process 3159 exited with code 0
2026/01/03 19:22:55 [notice] 3158#3158: signal 29 (SIGIO) received
2026/01/03 19:22:55 [notice] 3158#3158: signal 17 (SIGCHLD) received from 3160
2026/01/03 19:22:55 [notice] 3158#3158: worker process 3160 exited with code 0
2026/01/03 19:22:55 [notice] 3158#3158: exit
2026/01/03 19:22:55 [notice] 3878#3878: using the "epoll" event method
2026/01/03 19:22:55 [notice] 3878#3878: nginx/1.28.0
2026/01/03 19:22:55 [notice] 3878#3878: OS: Linux 6.1.158-180.294.amzn2023.x86_64
2026/01/03 19:22:55 [notice] 3878#3878: getrlimit(RLIMIT_NOFILE): 65535:65535
2026/01/03 19:22:55 [notice] 3923#3923: start worker processes
2026/01/03 19:22:55 [notice] 3923#3923: start worker process 3928
2026/01/03 19:22:55 [notice] 3923#3923: start worker process 3929
2026/01/03 19:54:00 [notice] 3923#3923: signal 3 (SIGQUIT) received from 1, shutting down
2026/01/03 19:54:00 [notice] 3928#3928: gracefully shutting down
2026/01/03 19:54:00 [notice] 3928#3928: exiting
2026/01/03 19:54:00 [notice] 3928#3928: exit
```

```
cat /var/log/nginx/access.log
```

- Save screenshot as: task7_access_log.png – content of access.log.

[illegible]

```
cat /etc/nginx/mime.types
```

- Save screenshot as: `task7_mime_types.png` – content of `mime.types`.


```
[ec2-user@ip-10-0-10-152 ~]$ cat /etc/nginx/mime.types
types {
application/A2L                                a2l;
application/AML                                aml;
application/andrew-inset                       ez;
application/ATF                                atf;
application/ATFX                               atfx;
application/ATXML                              atxml;
application/atom+xml                           atom;
application/atomcat+xml                       atomcat;
application/atomdeleted+xml                   atomdeleted;
application/atomsvc+xml                       atomsvc;
application/atsc-dwd+xml                     dwd;
application/atsc-held+xml                     held;
application/atsc-rsat+xml                     rsat;
application/auth-policy+xml                   apxml;
application/bacnet-xdd+zip                    xdd;
application/calendar+xml                     xcs;
application/cbor                               cbor;
application/ccex                               c3ex;
application/ccmp+xml                          ccmp;
application/ccxml+xml                         ccxml;
application/CDFX+XML                          cdfx;
application/cdmi-capability                   cdmia;
application/cdmi-container                    cdmic;
application/cdmi-domain                       cdmid;
application/cdmi-object                       cdmio;
application/cdmi-queue                        cdmiq;
application/CEA                               cea;
application/cellml+xml                        cellml cml;
application/clue_info+xml                     clue;
application/cms                               cmsc;
```

cat /etc/ssl/certs/selfsigned.crt

- Save screenshot as: task7_ssl_cert.png – content of selfsigned.crt.

```
[ec2-user@ip-10-0-10-152 ~]$ cat /etc/ssl/certs/selfsigned.crt
-----BEGIN CERTIFICATE-----
MIIDOzCCAiOgAwIBAgIUd9y+KR5TchNvZkmlDgQNJv10wewwDQYJKoZIhvcNAQEL
BQAwFjEUMBIGA1UEAwMLMy4yOC4yMDUuNDMwHhcNMjYwMTAzMTkyMjU1WhcNMjcw
MTAzMTkyMjU1WjAwMRQwEgYDVQDDAsZLjI4LjIwNS40MzCCASIwDQYJKoZIhvcN
AQEBBQADggEPADCCAQoCggEBAJXchwVjS/T+YichHZKeCwevaDGjatxwib+cq0T+m
GGPe7AoQ7sgy9Na6WID9cIwIkg+Go+aqiG7bXDwd7QyCz9V3JnRpIH0QHs0bh25Q
0LxL7d73pxJz4UF1mQ4MYzuUwp5nFAouh379t17FqeSVXTurfsDK9hu11BAG0Pwk0
7j/Ul6hZKwJuJYGrorORn2j0MM9w10F1eC1gKz8LEgMXZoL8m4WxSU83TojsRdr-i
OIjKEGaFnZuAPqGNYonzUxS9EZfGrCbF5Q1CMrFLLg4jLOjse+TVGK1623GNsbkr
nEf1SZM2b61grkPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPwPw
A1UdDgQwBBQrv50bvTcQvBkxnm6Nkrk2gqdjJjAfbgNvHSMEGDAWgBQrv50bvTcQ
vBkxnm6Nkrk2gqdjJjAPBgNVHREECDAghwQDHM0rMAKGA1UdEwQCMAAwCwYDVz0P
BAQDAgWgMBMGA1UdJQQMMAoGCCsGAQUFBwMBMA8GCsGGSIB3DQEBwUAA4IBAQAAb
bPArRk5fnagMxi1y2jXzrv5JqCF+a/y1XkjlcGOfwmkeUCfHdkeyJosrRYkfJeq
K3r4ks8w+GOQ0WoeaoJPk2TRdkVv01IMmqbBbZosYRW3IAT0pZVje0hH86vKcEnZ
JENktju0omo79ZpqzNzYLy12ja+Pk4fasJ8KTxkcUho5fqCoInyVlPn/4ItacvoF
ftWxTdIHmFjwNCZwMmdhQnNF11Er87vsjwq+HwdNAO1fX8Y05NKdd/J1jSaLIId8D
aLAbv6/1r0sYjsAhAdPPBTa1CPZDK5Xy40KiffAybbtvTydEf/fnqdKeZgJsY6Ps
imRrcLkNdND1o93gOmQC
-----END CERTIFICATE-----
[ec2-user@ip-10-0-10-152 ~]$
```

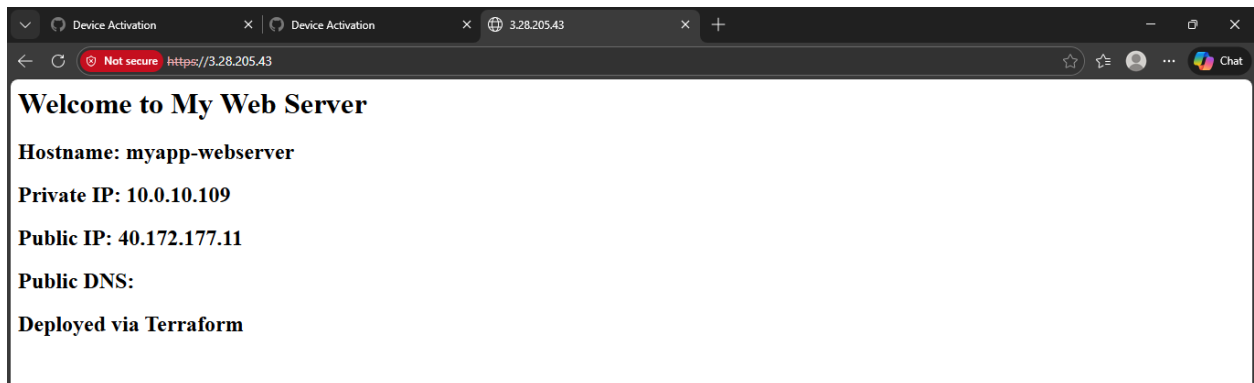
sudo cat /etc/ssl/private/selfsigned.key

- Save screenshot as: task7_ssl_key.png – content of selfsigned.key.


```
[ec2-user@ip-10-0-10-152 ~]$ sudo cat /etc/ssl/private/selfsigned.key
-----BEGIN PRIVATE KEY-----
MIIEVAIBADANBgkqhkiG9w0BAQEFAASCBCYwggSiAgEAAoIBAQCv3F1Y0v0/mInI
R2SngsHr2x02rccIm/nKtE/phhj3UwKE07IMvTWul1A/XCFopIPhQpMqohu21w1
ne0Mgs/VdyZ0aSBzkB7Dm4duUNC85+3e96cSc+FBZzk0DGM7lMkeZxQKLod+/bde
xanklV07n7AyvYbpdQ0Btd8JN04/1JeoKSScBib8q6K6ETdo9DDPVtThdXgtYCs/
CxIDF2aC/3uFsU1PN06I7EXa4jiIyhbMhZ2bgD6hjWkJ81MuvRGXcxqwmxeUNQjK3
yy40Iyo7Hvk1Ritettx1km5K5xH4kmTNm+tYK5D1mj8XoB+kIkz8wnGXETSYapi
RsEdh2mNagMBAAECggEAI/+1mse8PA2pc2nq/05FJSHQ/1Miz2haVz6K225Lx5ni
05zLr13GQNqsrfezkvZ1fbNLJfrrIS4inWkdcfbudAnW0/qQ6BHDwhhtXUiGpsSH0
11QIVz9s6ee894n0QHjAs4pxXuzEcYCY2YpEAHv81HjMu70Zd/U7DCm/RrnfT3+k
/Ma8DE2+mpLf/uD1UJ5J6nP01PzFymv87TaNPgNH/Mc2uB7p+fgAAkePQGZPk9b+
PmTGZL1NuZk/nrUBxHj0/3Ux1zzh30r22XJ1c3o57e1ib5NL+MywE7aVXIzdBMMy2
/qudu1/jCsok6cIg7I2i7GjZIIJHw3MvdKghykeoQKBgQDhUXYTL0QJgnhDWYFu
50kBBWGFmDertYuUmnEswQ2GRYMTFvGEmgkxpfZk7+22q1NvmyAz7h1DQ85m9is0
VGqwekdXay3eL56kzExRbYwdiFGXEGiab7cz4r8Qfa9JUhtb1JEVoScvvyg1WlyN
bBAPLHWA3ikmhyVIApJn/XiDOQKBgQDAelDmZk0Xv14hWEPRF7U6MZ5C72o3Gz/
vF4CbMTNbMqiVN+0LAiZP2VnzHk94RTiUvy3biKxs+T81h85h5tzH7sg3MjN3muf
aQcgwdmCm2ZmdHcmQghPeiUK+LDWdQ1Jd0bd0CwwC7Vdtrf97EKMIcaZ3XxyG187
XG0mxZx09QKBgGhd9Vpyn1Iqgaqpb61peLStwcrLt4MqXbD6Q61NC+fbuxuqr
hF8Uy+1mPYF0cXuhDo12+tJG+LNzih0P70tyf2+i1dPg0d1GsEIfpVEu33CIRW
npaitetwPYfKcGmD8omaZbI9t6sG/4z5YtjxdCyk2rwaZi/E8Ex3i9wa5AoGAf0v3
mS11tuMihFVnnRU9ib9T88yG7VgosRP01CbT51CxJjw7pQY8WRQ/Ha2VqNhSv8I
ORhLx5i8ugNgmJX5Qh1+wiERpqq8EUmd5nOfhA3LWPzn8EFzj3k0PB/qJBIQUkp
9uquIvr12DEIhx2rjktxdsBQFeUv/O/rHY/VlHECgYBuddtWwVxK61T8QUdzhj0In
4WQRkgtxzKlw08iM1nt/LCub9ruBjd9/6zNGX8WwpHxUaPN1EoFYJ+ZPRKW9+0Rp
og/xC0dTOBBRQz3MPNm/TrkAouVxbJ947pYwW3GTxiha0ZOmIwF7QtJLCVRvBM9
7tjAB8K+wt5TGvK07VvMhQ==
-----END PRIVATE KEY-----
[ec2-user@ip-10-0-10-152 ~]$
```

10. Test reverse proxy in browser:

- Open browser and navigate to <https://<webserver-public-ip>>
- You should see the web-1 Apache page through the Nginx proxy
- Save screenshot as: task7_reverse_proxy_browser.png — browser showing web-1 content through proxy.



Task 8 – Configure Nginx as load balancer

In this task, you will add a second backend server and configure Nginx to load balance between them.

1. Add the second web server module to main.tf:

```

module "myapp-web-2" {
  source = "./modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var. availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_id = module.myapp-subnet.subnet.id
  script_path = "./apache.sh"
}
  
```

```
    instance_suffix = "2"
}
```

- **Save screenshot as:** task8_main_tf_web2.png – main.tf showing myapp-web-2 module.

```
module "myapp-web-1" {
  source          = "../modules/webserver"
  env_prefix      = var.env_prefix
  instance_type   = var.instance_type
  availability_zone = var.availability_zone
  public_key      = var.public_key
  my_ip           = local.my_ip
  vpc_id          = aws_vpc.myapp_vpc.id
  subnet_id       = module.myapp-subnet.subnet_id
  script_path     = "./apache.sh"
  instance_suffix = "1"
  ami_id          = data.aws_ami.amazon_linux.id
}

module "myapp-web-2" {
  source          = "../modules/webserver"
  env_prefix      = var.env_prefix
  instance_type   = var.instance_type
  availability_zone = var.availability_zone
  public_key      = var.public_key
  my_ip           = local.my_ip
  vpc_id          = aws_vpc.myapp_vpc.id
  subnet_id       = module.myapp-subnet.subnet_id
  script_path     = "./apache.sh"
  instance_suffix = "2"
}
```

2. Update outputs.tf:

```
output "aws_web-2_public_ip" {
  value = module.myapp-web-2.aws_instance.public_ip
}
```

- **Save screenshot as:** task8_outputs_web2.png – outputs.tf with web-2 output.

```
GNU nano 7.2 outputs.tf
output "webserver_public_ip" {
  value = module.myapp-webserver.aws_instance.public_ip
}
output "aws_web-1_public_ip" {
  value = module.myapp-web-1.aws_instance.public_ip
}
output "aws_web-2_public_ip" {
  value = module.myapp-web-2.aws_instance.public_ip
}
```

3. Apply the configuration:

terraform apply -auto-approve

- **Save screenshot as:** task8_terraform_apply.png – terraform apply output showing web-2 created.

```

module.myapp-subnet.aws_default_route_table.main_rt: Creation complete after 0s [id=rtb-0d08a7afb35401afb]
module.myapp-web-2.aws_security_group.web_sg: Creation complete after 3s [id=sg-0a2b7a3a56aa32520]
module.myapp-web-1.aws_security_group.web_sg: Creation complete after 3s [id=sg-00b3d13590986afe1]
module.myapp-webserver.aws_security_group.web_sg: Creation complete after 3s [id=sg-046a91dc77356be73]
module.myapp-subnet.aws_subnet.subnet: Still creating... [00m10s elapsed]
module.myapp-subnet.aws_subnet.subnet: Creation complete after 11s [id=subnet-047fd2dcfe91fbcf8]
module.myapp-web-2.aws_instance.myapp-server: Creating...
module.myapp-web-1.aws_instance.web: Creating...
module.myapp-web-2.aws_instance.web: Creating...
module.myapp-webserver.aws_instance.myapp-server: Creating...
module.myapp-web-1.aws_instance.myapp-server: Creating...
module.myapp-webserver.aws_instance.web: Creating...
module.myapp-web-1.aws_instance.web: Still creating... [00m10s elapsed]
module.myapp-web-2.aws_instance.myapp-server: Still creating... [00m10s elapsed]
module.myapp-webserver.aws_instance.myapp-server: Still creating... [00m10s elapsed]
module.myapp-web-2.aws_instance.web: Still creating... [00m10s elapsed]
module.myapp-web-1.aws_instance.myapp-server: Still creating... [00m10s elapsed]
module.myapp-webserver.aws_instance.web: Still creating... [00m10s elapsed]
module.myapp-webserver.aws_instance.web: Creation complete after 13s [id=i-0db7b9c122bceaec0]
module.myapp-web-1.aws_instance.myapp-server: Creation complete after 13s [id=i-037f92577ed0f15f4]
module.myapp-web-2.aws_instance.myapp-server: Creation complete after 13s [id=i-0cecee43e9ca97a73]
module.myapp-web-1.aws_instance.web: Creation complete after 13s [id=i-084ac20b45afed4b0]
module.myapp-webserver.aws_instance.myapp-server: Creation complete after 13s [id=i-05c051e1a83401543]
module.myapp-web-2.aws_instance.web: Creation complete after 13s [id=i-010581e85d67c3d2e]

```

Apply complete! Resources: 14 added, 0 changed, 0 destroyed.

Outputs:

```

aws_web-1_public_ip = "3.29.58.146"
aws_web-2_public_ip = "51.112.48.152"
aws_webserver_public_ip = "3.29.30.34"

```

@Hamail-maker → /workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) \$

4. Get all outputs:

terraform output

- Save screenshot as: task8_terraform_output.png — showing all three public IPs.

```

@Hamail-maker → /workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ terraform output
aws_web-1_public_ip = "3.29.58.146"
aws_web-2_public_ip = "51.112.48.152"
aws_webserver_public_ip = "3.29.30.34"
@Hamail-maker → /workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $

```

5. SSH into the webserver (Nginx proxy):

ssh ec2-user@<webserver-public-ip>

6. Edit Nginx configuration for load balancing:

sudo vim /etc/nginx/nginx.conf

Update the upstream block and location:

```

upstream backend_servers {
    server <web-1-public-ip>:80;
    server <web-2-public-ip>:80;
}

# ... in server block:
location / {
    # root /usr/share/nginx/html;
    # index index.html;
    # proxy_pass http://<web-1-public-ip>:80;
    proxy_pass http://backend_servers;
}

```

- Save screenshot as: task8_nginx_conf_load_balancer.png — nginx.conf showing load balancing configuration.

```

sendfile on;
tcp_nopush on;
keepalive_timeout 65;
types_hash_max_size 4096;

include /etc/nginx/mime.types;
default_type application/octet-stream;

# 🔄 Load Balancer Backend Pool
upstream backend_servers {
    server 3.29.58.146:80;
    server 51.112.48.152:80;
}

# 🛡️ HTTPS Server (Load Balancer)
server {
    listen 443 ssl;
    server_name 3.29.30.34;

    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    location / {
        proxy_pass http://backend_servers;
    }
}

# 🔄 HTTP → HTTPS Redirect
server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}

```

"/etc/nginx/nginx.conf" 50L, 1170B

7. Restart Nginx:

sudo systemctl restart nginx

- **Save screenshot as:** task8_nginx_restart.png – terminal showing nginx restart.

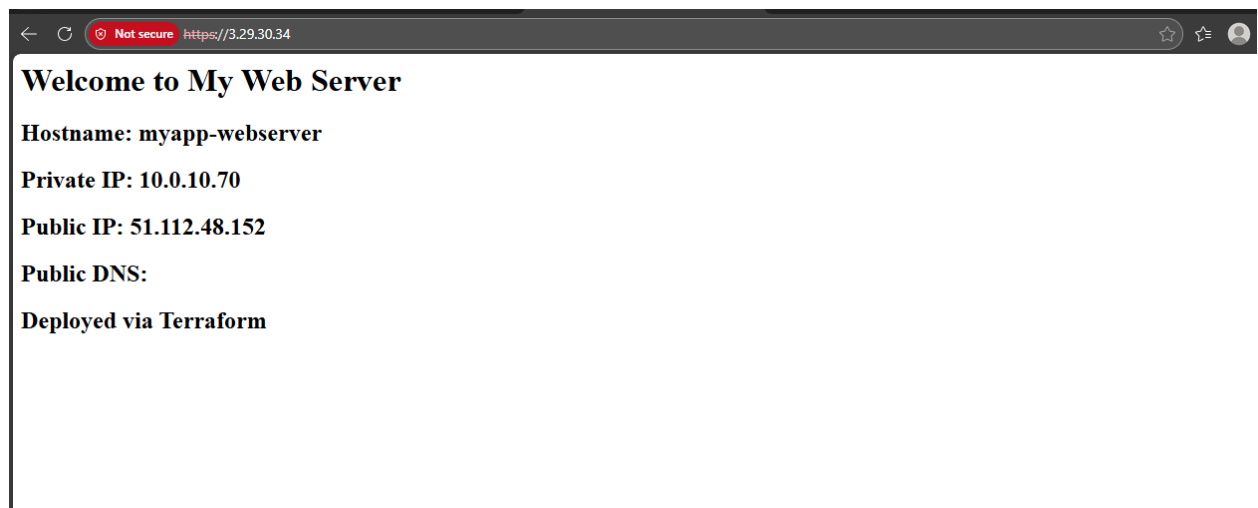
```

[ec2-user@ip-10-0-10-106 ~]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-10-0-10-106 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-106 ~]$

```

8. Test load balancing in browser:

- Open browser and navigate to https://<webserver-public-ip>
- Reload the page multiple times
- You should see the content alternating between web-1 and web-2 (check the hostname/IP in the page)
- **Save screenshot as:** task8_load_balancer_web1.png – browser showing web-1 content.



- **Save screenshot as:** task8_load_balancer_web2.png – browser showing web-2 content after reload.

Welcome to My Web Server

Hostname: myapp-webserver

Private IP: 10.0.10.132

Public IP: 3.29.58.146

Public DNS:

Deployed via Terraform

Task 9 – Configure high availability with backup servers

In this task, you will configure one server as primary and another as backup for high availability.

1. SSH into the webserver:

ssh ec2-user@<webserver-public-ip>

2. Edit Nginx configuration for high availability:

sudo vim /etc/nginx/nginx.conf

Update the upstream block to make web-2 a backup:

```
upstream backend_servers {
    server <web-1-public-ip>:80;
    server <web-2-public-ip>:80 backup;
}
```

- **Save screenshot as:** task9_nginx_conf_ha_web1_primary.png – nginx.conf with web-2 as backup.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
--
sendfile on;
tcp_nopush on;
keepalive_timeout 65;
types_hash_max_size 4096;

include /etc/nginx/mime.types;
default_type application/octet-stream;

# Load Balancer Backend Pool
upstream backend_servers {
    server 3.29.58.146:80;
    server 51.112.48.152:80 backup;
}

# HTTPS Server (Load Balancer)
server {
    listen 443 ssl;
    server_name 3.29.30.34;

    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    location / {
        proxy_pass http://backend_servers;
    }
}

# HTTP to HTTPS Redirect
server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}

-- INSERT --
28.40

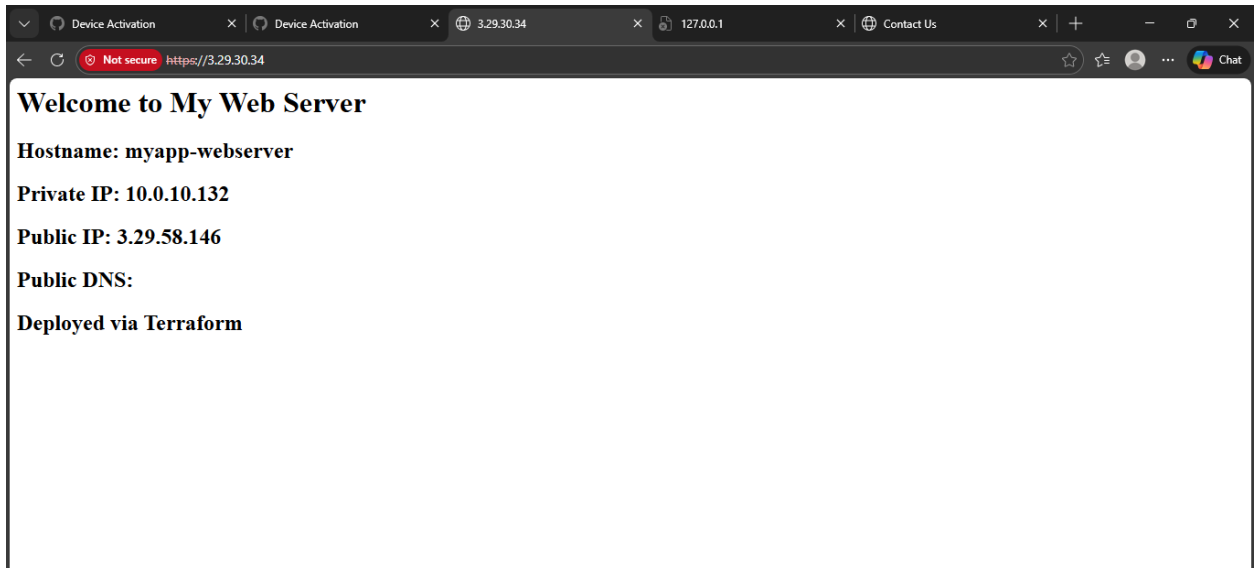
```

3. Restart Nginx:

`sudo systemctl restart nginx`

4. Test in browser:

- Open browser and navigate to `https://<webserver-public-ip>`
- Reload multiple times
- You should ONLY see web-1 (primary server)
- **Save screenshot as:** `task9_ha_web1_only.png` – browser showing only web-1 content on multiple reloads.



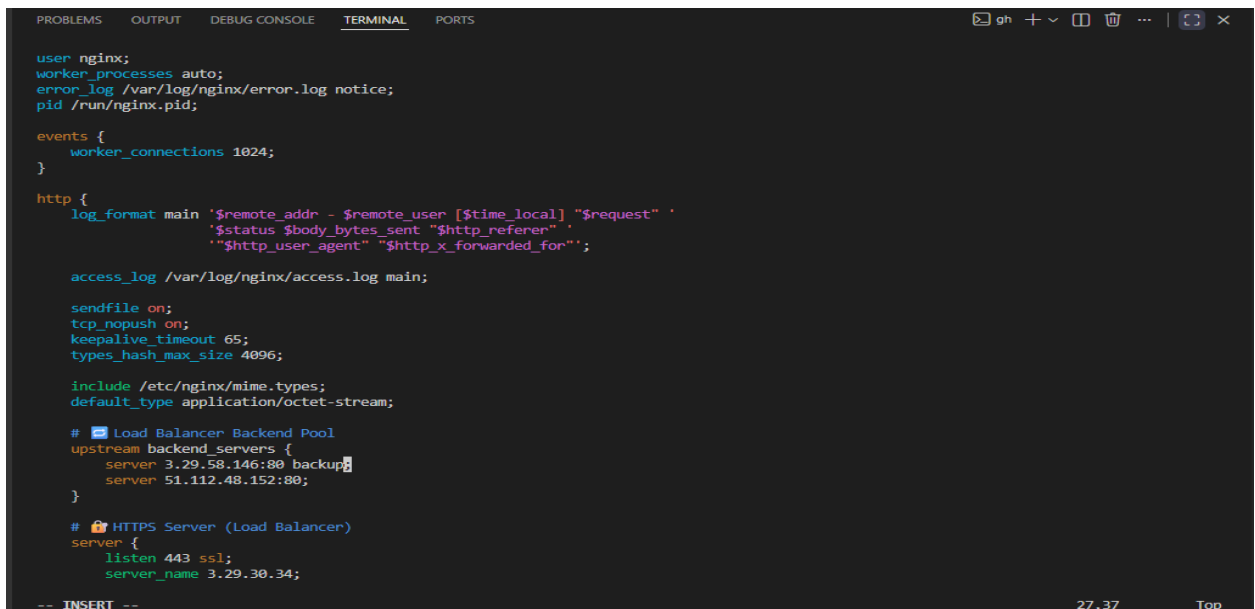
5. Switch backup configuration:

`sudo vim /etc/nginx/nginx.conf`

Update to make web-1 backup:

```
upstream backend_servers {
    server <web-1-public-ip>:80 backup;
    server <web-2-public-ip>:80;
}
```

- **Save screenshot as:** `task9_nginx_conf_ha_web2_primary.png` – `nginx.conf` with web-1 as backup.

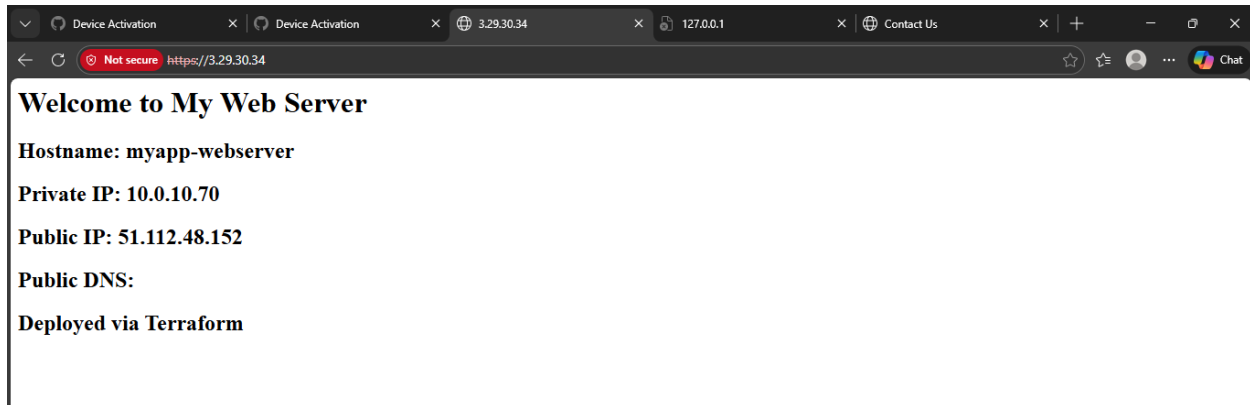


6. Restart Nginx:

`sudo systemctl restart nginx`

7. Test in browser:

- Reload multiple times
- You should ONLY see web-2 (now the primary server)
- **Save screenshot as:** task9_ha_web2_only.png – browser showing only web-2 content on multiple reloads.



Task 10

In this task, you will enable caching in Nginx to improve performance.

1. SSH into the webserver:

`ssh ec2-user@<webserver-public-ip>`

2. Edit Nginx configuration to enable caching:

`sudo vim /etc/nginx/nginx.conf`

Add proxy cache configuration in the http block and location block:

```
http {  
    proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=my_cache:10m inactive=60m  
    max_size=1g;
```

```
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '  
        '$status $body_bytes_sent "$http_referer" '  
        '"$http_user_agent" "$http_x_forwarded_for";
```

... other settings ...

```
upstream backend_servers {  
    server <web-1-public-ip>:80;  
    server <web-2-public-ip>:80;  
}
```

```
server {  
    listen 443 ssl;  
    server_name $PUBLIC_IP;  
    ssl_certificate /etc/ssl/certs/selfsigned.crt;  
    ssl_certificate_key /etc/ssl/private/selfsigned.key;
```

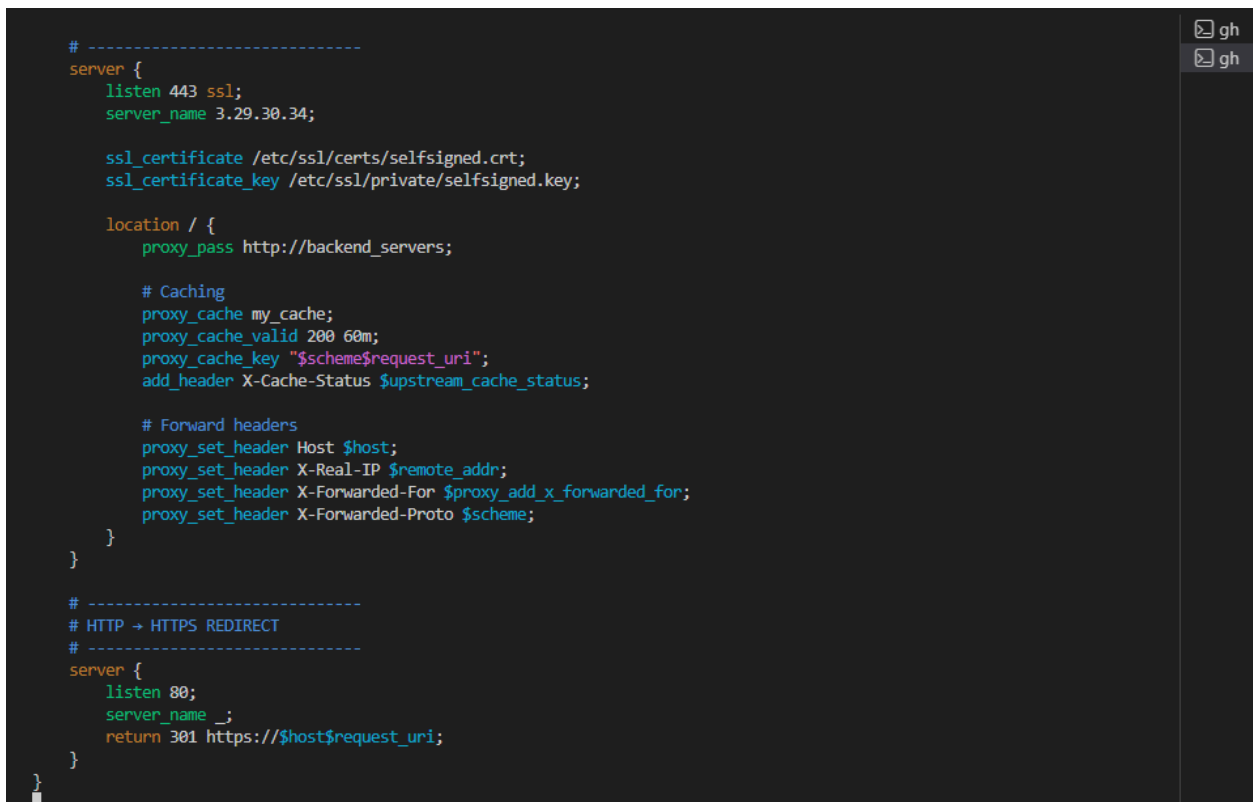
```
    location / {  
        # root /usr/share/nginx/html;  
        # index index.html;  
        # proxy_pass http://<web-1-public-ip>:80;
```

- ```

 proxy_pass http://backend_servers;
 proxy_cache my_cache;
 proxy_cache_valid 200 60m;
 proxy_cache_key "$scheme$request_uri";
 add_header X-Cache-Status $upstream_cache_status;
 }
}

... rest of config ...
}

```
- **Save screenshot as:** task10\_nginx\_conf\_cache.png – nginx.conf showing cache configuration.



```

server {
 listen 443 ssl;
 server_name 3.29.30.34;

 ssl_certificate /etc/ssl/certs/selfsigned.crt;
 ssl_certificate_key /etc/ssl/private/selfsigned.key;

 location / {
 proxy_pass http://backend_servers;

 # Caching
 proxy_cache my_cache;
 proxy_cache_valid 200 60m;
 proxy_cache_key "$scheme$request_uri";
 add_header X-Cache-Status $upstream_cache_status;

 # Forward headers
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 }
}

HTTP → HTTPS REDIRECT

server {
 listen 80;
 server_name _;
 return 301 https://$host$request_uri;
}
}

```

- ### 3. Restart Nginx:
- ```
sudo systemctl restart nginx
```
- **Save screenshot as:** task10_nginx_restart.png – terminal showing nginx restart.


```

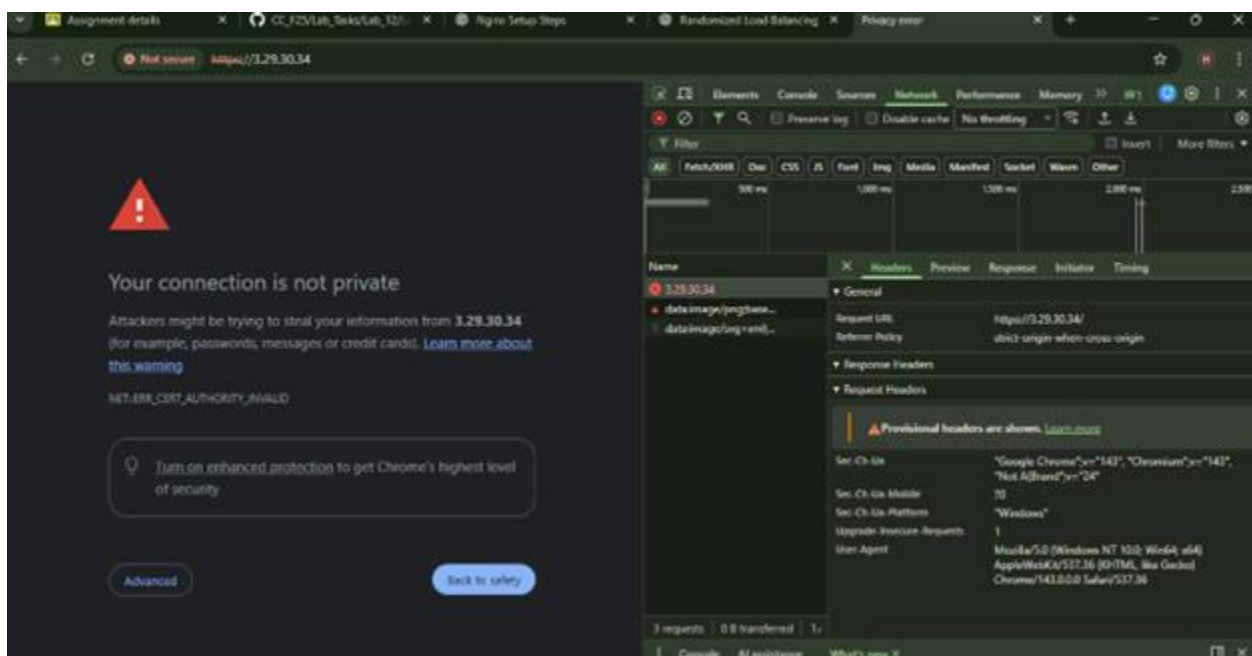
Last login: Mon Jan 5 07:45:59 2026 from 4.240.39.195
[ec2-user@ip-10-0-10-106 ~]$ sudo vim /etc/nginx/nginx.conf
[ec2-user@ip-10-0-10-106 ~]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-10-0-10-106 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-106 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-01-05 08:02:23 UTC; 8s ago
     Process: 31288 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 31289 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 31290 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
    Main PID: 31291 (nginx)
      Tasks: 5 (limit: 1067)
     Memory: 4.4M
        CPU: 62ms
    CGroup: /system.slice/nginx.service
            └─31291 "nginx: master process /usr/sbin/nginx"
              └─31292 "nginx: worker process"
                └─31293 "nginx: worker process"
                  └─31294 "nginx: cache manager process"
                    └─31295 "nginx: cache loader process"

Jan 05 08:02:23 ip-10-0-10-106.me-central-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server:
Jan 05 08:02:23 ip-10-0-10-106.me-central-1.compute.internal nginx[31289]: nginx: the configuration file /etc/nginx/nginx.conf is ok
Jan 05 08:02:23 ip-10-0-10-106.me-central-1.compute.internal nginx[31289]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Jan 05 08:02:23 ip-10-0-10-106.me-central-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server:
lines 1-21/21 (END)

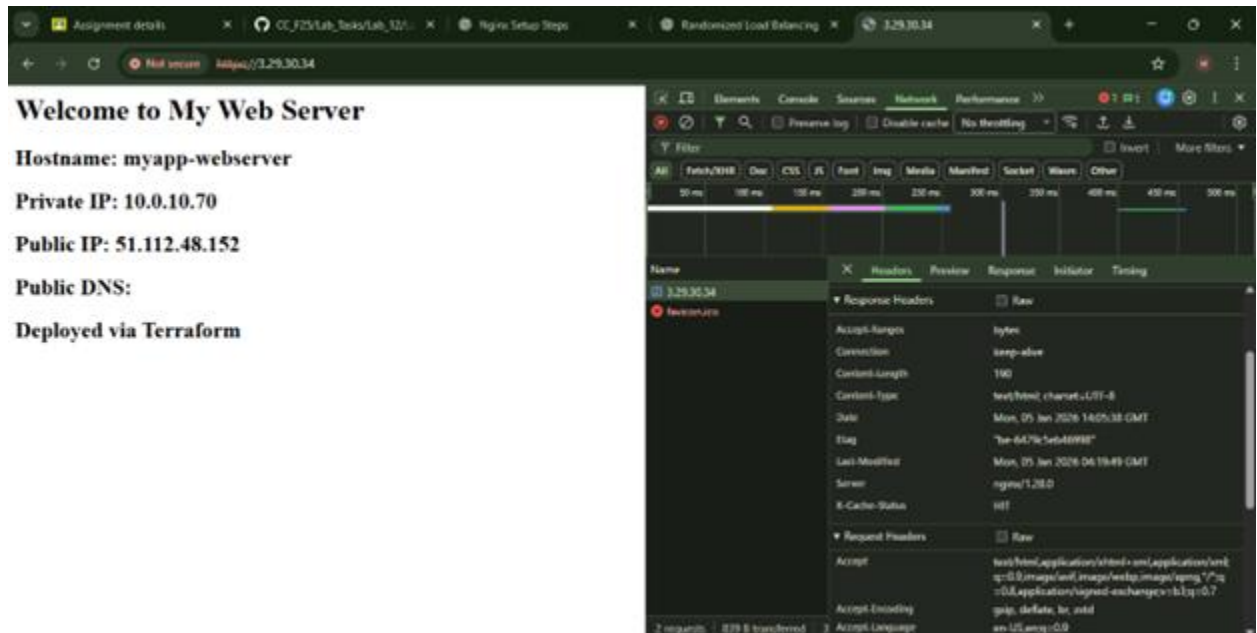
```

4. Test caching in browser:

- Open browser developer tools (F12)
- Navigate to Network tab
- Visit <https://<webserver-public-ip>>
- Check response headers for X-Cache-Status
- First request should show MISS
- Reload the page
- Second request should show HIT
- **Save screenshot as:** task10_cache_miss.png – browser dev tools showing X-Cache-Status: MISS on first request.



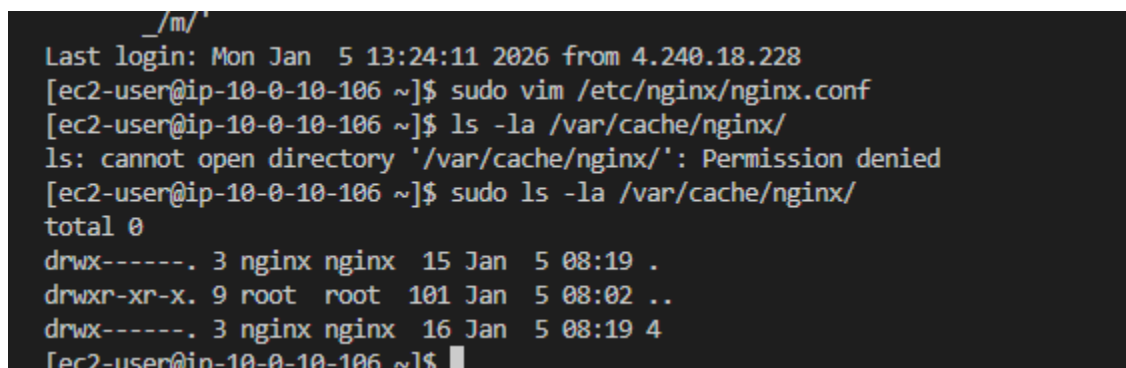
- **Save screenshot as:** task10_cache_hit.png – browser dev tools showing X-Cache-Status: HIT on subsequent request.



5. Verify cache directory:

`ls -la /var/cache/nginx/`

- **Save screenshot as:** task10_cache_directory.png – terminal showing cache directory contents.



Cleanup

1. Exit SSH session:
 - `exit`
 2. Destroy all resources:
 - `terraform destroy`
- Type yes when prompted for confirmation.
 - Save screenshot as: cleanup_destroy_prompt.png – terminal showing terraform destroy prompt.

```
Plan: 0 to add, 0 to change, 14 to destroy.
```

```
Changes to Outputs:
```

```
- aws_web-1_public_ip      = "3.29.58.146" -> null
- aws_web-2_public_ip      = "51.112.48.152" -> null
- aws_webserver_public_ip  = "3.29.30.34" -> null
```

```
Do you really want to destroy all resources?
```

```
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.
```

```
Enter a value: 
```

- Save screenshot as: cleanup_destroy_complete.png – terraform destroy completion output.

```
module.myapp-subnet.aws_subnet.subnet: Destroying... [id=subnet-047fd2dcfe91fbcf8]
aws_key_pair.web_key: Destruction complete after 1s
module.myapp-subnet.aws_subnet.subnet: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-049e7547fe6199c29]
aws_vpc.myapp_vpc: Destruction complete after 1s
```

```
Destroy complete! Resources: 14 destroyed.
```

```
@hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

3. Verify state files:
cat terraform.tfstate
- Save screenshot as: cleanup_state_empty.png – showing empty terraform. tfstate.

```
@hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 231,
  "lineage": "ae7b97d4-f405-2014-da24-930c183c6be9",
  "outputs": {},
  "resources": [],
  "check_results": null
}
@hamail-maker →/workspaces/CC_Hamail-Fatima_2023-BSE-023_Lab12 (main) $
```

4. List all project files:
tree
or
ls -la
- Save screenshot as: cleanup_final_files.png – showing final project structure.

```
├── LICENSE.txt
├── METADATA
├── RECORD
├── REQUESTED
├── WHEEL
├── direct_url.json
├── entry_points.txt
├── install
├── awscli2.zip
├── entry-script.sh
├── entry-scripts.sh
├── locals.tf
├── main.tf
├── modules
│   ├── subnet
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   └── webserver
│       ├── main.tf
│       ├── outputs.tf
│       └── variables.tf
├── outputs.tf
├── terraform-key.pub
├── terraform.tfstate
├── terraform.tfstate.backup
├── terraform.tfvars
├── terraform_1.14.3_linux_amd64.zip
└── variables.tf
```

1088 directories, 7615 files

@hamail-maker → /workspaces/CC Hamail-Fatima 2023-BSE-023 Lab12 (main) \$